Modular and Scenario-Based Cybersecurity Curriculum with Real-Time Threat Intelligence Integration

A Capstone Project Report submitted to

Mohan Babu University, Tirupati

In partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Poleboina divya	**22102A041020**
S Mohammed Zaki	**22102A041068**
S muhammad Akram	**22102A041062**
S pavana sutha Nithin	**22102A041040**

Under the Esteemed Supervision of

Mr. N. Purushottam
Assistant Professor
Department of CSE

Department of Computer Science and Engineering

School of Computing

Mohan Babu University

Sree Sainath Nagar, Tirupati – 517 102

2025 – 2026

MOHAN BABU UNIVERSITY
Sree Sainath Nagar, Tirupati 517 102

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Certificate

This is to certify that the Capstone Project report titled **"Modular and Scenario-Based Cybersecurity Curriculum with Real-Time Threat Intelligence Integration"** is the Bonafide work done by

Poleboina divya	**22102A041020**
S Mohammed Zaki	**22102A041068**
S muhammad Akram	**22102A041062**
S pavana sutha Nithin	**22102A041040**

in the Department of **Computer Science and Engineering**, and submitted to Mohan Babu University, Tirupati in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** during the academic year 2025-2026.

This is work has been carried out under my guidance and supervision.

The results embodied in this project report have not been submitted in any University or Organization for the award of any degree or diploma.

Supervisor:

Mr. N. Purushottam
Assistant Professor
Dept. of CSE

Head:

Dr. G. Sunitha
Professor & Head
Dept. of CSE

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We are extremely thankful to our beloved Chancellor, **Dr. M. Mohan Babu garu**, for his inspiration and unwavering commitment to academic growth and student development.

We express sincere gratitude to our Pro-Chancellor, **Sri Vishnu Manchu garu**, his constant encouragement and visionary leadership in fostering a culture of learning and excellence.

We express sincere gratitude to our Honorable Vice-Chancellor, **Prof. Nagaraj Ramrao garu**, for his valuable guidance, encouragement, and continuous support in all academic endeavours.

We are highly indebted to **Dr. Babu Devasenapathy**, Dean-School of Engineering & Computing for his valuable support and guidance in all the academic matters.

We are deeply obliged to **Dr. G. Sunitha**, Professor & Head, Department of CSE, for her motivation, insightful guidance, and persistent encouragement.

We would like to extend heartfelt thanks to the supervisor, **Mr. N. Purushottam**, Assistant Professor, Department of CSE, for the consistent support, insightful guidance, and constructive feedback provided during the project work.

Poleboina divya	**22102A041020**
S Mohammed Zaki	**22102A041068**
S muhammad Akram	**22102A041062**
S pavana sutha Nithin	**22102A041040**

DECLARATION

We hereby declare that this capstone project report titled **"Modular and Scenario-Based Cybersecurity Curriculum with Real-Time Threat Intelligence Integration"** is a genuine project work carried out by us, in **B.Tech.** ***(Computer Science and Engineering)*** degree course of **Mohan Babu University, Tirupati** and has not been submitted to any other course or University for the award of any degree by us.

Poleboina divya	**22102A041020**
S Mohammed Zaki	**22102A041068**
S muhammad Akram	**22102A041062**
S pavana sutha Nithin	**22102A041040**

Vision

To be a globally respected institution with an innovative and entrepreneurial culture that offers transformative education to advance sustainability and societal good.

Mission

- Develop industry-focused professionals with a global perspective.
- Offer academic programs that provide transformative learning experience founded on the spirit of curiosity, innovation, and integrity.
- Create confluence of research, innovation, and ideation to bring about sustainable and socially relevant enterprises.
- Uphold high standards of professional ethics leading to harmonious relationship with environment and society.

SCHOOL OF COMPUTING

Vision

To lead the advancement of computer science research and education that has real-world impact and to push the frontiers of innovation in the field.

Mission

- Instil within our students fundamental computing knowledge, a broad set of skills, and an inquisitive attitude to create innovative solutions to serve industry and community.
- Provide an experience par excellence with our state-of-the-art research, innovation, and incubation ecosystem to realise our learners' fullest potential.
- Impart continued education and research support to working professionals in the computing domain to enhance their expertise in the cutting-edge technologies.
- Inculcate among the computing engineers of tomorrow with a spirit to solve societal challenges.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision

To become a Centre of Excellence in Computer Science and its emerging areas by imparting high quality education through teaching, training and research.

Mission

- Imparting quality education in Computer Science and Engineering and emerging areas of IT industry by disseminating knowledge through contemporary curriculum, competent faculty and effective teaching-learning methodologies.
- Nurture research, innovation and entrepreneurial skills among faculty and students to contribute to the needs of industry and society.
- Inculcate professional attitude, ethical and social responsibilities for prospective and promising engineering profession.
- Encourage students to engage in life-long learning by creating awareness of the contemporary developments in Computer Science and Engineering and its emerging areas.

B.Tech. Computer Science and Engineering

PROGRAM EDUCATIONAL OBJECTIVES

After few years of graduation, the graduates of B.Tech. CSE will be:

1. Pursuing higher studies in core, specialized or allied areas of Computer Science, or Management.
2. Employed in reputed Computer and I.T organizations or Government to have a globally competent professional career in Computer Science and Engineering domain or be successful Entrepreneurs.
3. Able to demonstrate effective communication, engage in teamwork, exhibit leadership skills and ethical attitude, and achieve professional advancement through continuing education.

PROGRAM OUTCOMES

On successful completion of the Program, the graduates of B.Tech. CSE Program will be able to:

1. **Engineering Knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem Analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions

using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/Development of Solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct Investigations of Complex Problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern Tool Usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The Engineer and Society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and Sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and Team Work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project Management and Finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long Learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

On successful completion of the Program, the graduates of B. Tech. (CSE) program will be able to:

PSO1. Apply knowledge of computer science engineering, Use modern tools, techniques and technologies for efficient design and development of computer-based systems for complex engineering problems.

PSO2. Design and deploy networked systems using standards and principles, evaluate security measures for complex networks, apply procedures and tools to solve networking issues.

PSO3. Develop intelligent systems by applying adaptive algorithms and methodologies for solving problems from inter-disciplinary domains.

PSO4. Apply suitable models, tools and techniques to perform data analytics for effective decision making.

Course Code	Course Title	L	T	P	S	C
22CS108001	**Capstone Project**	----------	----------	----------	----------	10

COURSE OUTCOMES: After successful completion of this course, the students will be able to:

CO1. Create/Design computer science engineering systems or processes to solve complex computer science engineering and allied problems using appropriate tools and techniques following relevant standards, codes, policies, regulations and latest developments.

CO2. Consider society, health, safety, environment, sustainability, economics and project management in solving complex computer science engineering and allied problems.

CO3. Perform individually or in a team besides communicating effectively in written, oral and graphical forms on computer science engineering systems or processes.

CO-PO-PSO Mapping Table:

Cou rs e Outcom e	Program Outcomes												Program Specific Outcomes			
	PO 1	P O2	P O3	P O4	P O5	P O6	P O7	P O8	P O9	PO 10	PO 11	PO 12	PS O1	PS O2	PS O3	PS O4
CO1	3	3	3	3	3			3				3				
CO2						3	3				3					
CO3									3	3						

Correlation Level: 3-High; 2-Medium; 1-Low

TABLE OF CONTENTS

Chapter	Page no
Abstract	**1**
List of Acronyms	**2**
List of Figures	**3**
List of Tables	**4**
Chapter 1 Introduction	**5-8**
12.1 Introduction	**5**
12.2 Problem Statement	**6**
12.3 Objectives	**6**
12.4 Scope	**7**
12.5 Applications	**7**
12.6 Limitations	**8**
Chapter 2 Literature Survey	**9-14**
Chapter 3 Analysis & Design	**15-30**
3.1 Existing System	**15**
3.2 Disadvantages of Existing System	**16**
3.3 Proposed System	**17**
3.4 Advantages of Proposed System	**18**
3.5 Modules	**18-20**
3.6 Algorithms	**20-22**
3.7 System Architecture	**22-23**
3.8 UML Diagrams	**23-30**
Chapter 4 Implementation	**31-41**
4.1 Overview of Implementation	**31**
4.2 Hardware Requirements	**31**
4.3 Software Requirements	**32**
4.4 Functional & Non-Functional Requirements	**32**
4.5 Technologies Used	**32-34**
4.6 Module-wise Implementation	**34-35**
4.7 Data Handling	**35-36**

Chapter	Page no
4.8 Security Implementation	**37-38**
4.9 System Integration & Communication	**38-39**
4.10 System Performance Optimization	**39-41**
Chapter 5 Execution Procedure and Testing	**42-57**
5.1 Environment Setup	**42-43**
5.2 Code Execution	**44-45**
5.3 System workflow Execution	**45-46**
5.4 Introduction to Testing	**47-48**
5.5 Testing Objectives	**48-49**
5.6 White-Box Testing	**49-50**
5.7 Black-Box Testing	**50-51**
5.8 Unit Testing	**51-52**
5.9 Integration Testing	**52-53**
5.10 Security Testing	**54-55**
5.11 Validation Testing	**55-56**
5.12 Test Cases	**56-57**
Chapter 6 Results & Performance Evaluation	**58-73**
6.1 Results with Description	**58-65**
6.2 Performance Evaluation	**65-73**
Chapter 7 Conclusions and Future Work	**74-75**
7.1 Conclusion	**74**
7.2 Future Work	**75**
Appendix	
A. Sample Program Code	**76-96**
References	**97-98**

ABSTRACT

Traditional ride-sharing platforms, such as Uber and Lyft, rely on centralized architectures that grant a single entity total control over user data, ride logistics, and financial transactions. This centralization introduces significant vulnerabilities, including exorbitant intermediary fees, compromised data privacy, and a single point of failure that threatens system reliability. To address these challenges, this paper proposes a decentralized, peer-to-peer (P2P) ride-sharing ecosystem powered by private blockchain technology. By implementing the Proof of Authority (PoA) consensus algorithm, the system achieves high-speed transaction verification and low latency, meeting the real-time demands of urban mobility without the high computational costs of public blockchains.

The proposed framework introduces a login-free booking mechanism and decentralized identity verification, ensuring maximum user anonymity and autonomy. A core innovation of the system is the multi-transactional trip state machine, which utilizes smart contract-based escrow payments to secure funds until service completion. Furthermore, an immutable, tamper-proof reputation ledger is integrated directly into the blockchain, utilizing SHA-256 cryptographic hashing to ensure that ratings and driver histories remain verifiable and transparent. By eliminating the middleman, this decentralized solution reduces operational costs, enhances data integrity, and fosters a trustless environment for drivers and passengers alike. Experimental results demonstrate that the system provides a scalable, secure, and community-driven foundation for the next generation of smart mobility.

Keywords— Decentralized, Blockchain, Ride-Sharing, Peer-to-Peer, Proof of Authority, Smart Contract, Escrow, Reputation System, Token Payments.

List of Acronyms

Acronym	Definition
SHA	Secure Hash Algorithm
SQL	Structured Query Language
DBMS	Database Management System
IDE	Integrated Development Environment
API	Application Programming Interface
HTTP	HyperText Transfer Protocol Secure
TCP/IP	Transmission Control Protocol / Internet Protocol
DApp	Decentralized Application
XAI	Explainable Artificial Intelligence
SIEM	Security Information and Event Management
ACL	Access Control List

List of Figures

Figure No.	Description	Page No.
3.5	Modules Used in the System	19
3.7	System Architecture	23
3.8.1	Use Case Diagram	24
3.8.2	Class Diagram	25
3.8.3	Sequence Diagram	26
3.8.4	Activity Diagram	27
3.8.5	Collaboration Diagram	28
3.8.6	Deployment Diagram	29
3.8.7	ER diagram	30
5.2.1	Installing Dependencies	44
5.2.2	Running the Application	44
6.1.1	Student Registration	59
6.1.2	Student Login	60
6.1.3	Admin Console	61
6.1.4	Student Learning Console	61
6.1.5	Video-Based Learning	62
6.1.6	Hands-on Learning	63
6.1.7	Scenario-Based Threat Simulation	64

List of Tables

Table No.	Description	Page No.
5.12	Test Cases	57
6.2	Performance Evaluation	66

CHAPTER 1
INTRODUCTION

1.1. Introduction:

The rapid growth of urbanization and digital technologies has significantly transformed the transportation sector, making ride-sharing services an essential part of modern mobility. Platforms such as Uber and Lyft have revolutionized the way people commute by providing convenient, on-demand transportation services through mobile applications. However, these systems are built on centralized architectures, where a single organization controls user data, pricing mechanisms, and transaction processing. While this model offers ease of use, it introduces several critical challenges, including high commission fees, lack of transparency, privacy concerns, and dependence on a central authority. These limitations often result in reduced trust among users and drivers, as well as inefficiencies in service delivery.

One of the major drawbacks of centralized ride-sharing platforms is the lack of data ownership and control for users. Personal information, ride history, and payment details are stored and managed by a central entity, making them vulnerable to data breaches, misuse, and unauthorized access. Additionally, drivers often face unfair commission structures and sudden policy changes, while passengers have limited visibility into pricing algorithms and rating systems. Furthermore, the reliance on centralized servers creates a single point of failure, which can lead to service disruptions and reduced system reliability.

To address these challenges, the proposed project introduces a Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology. This system eliminates intermediaries by enabling direct interaction between riders and drivers through a secure and transparent decentralized network. The core of the system is built on a private blockchain that utilizes the Proof of Authority (PoA) consensus mechanism, ensuring efficient transaction validation with minimal computational overhead. Unlike traditional public blockchains, PoA offers faster processing speeds and lower costs, making it suitable for real-time applications such as ride-sharing.

The proposed system incorporates advanced features such as escrow-based payments, which securely hold funds until the completion of a ride, ensuring fairness for both parties. Additionally, a tamper-proof reputation system is implemented to maintain trust by storing user ratings on the blockchain, making them immutable and verifiable. The platform also supports login-free access, enhancing user convenience while preserving privacy. By leveraging the key characteristics of

blockchain—immutability, transparency, and decentralization—the system aims to provide a secure, efficient, and user-centric solution for modern ride-sharing challenges.

1.2. Problem Statement:

The rapid adoption of ride-sharing platforms has transformed urban transportation by offering convenient and cost-effective mobility solutions. However, most existing ride-sharing services operate on centralized systems, where a single organization controls all aspects of the platform, including user data, pricing, and transaction management. This centralized approach introduces several critical challenges that affect both riders and drivers.

One of the primary issues is the lack of data privacy and security. Users are required to share personal and financial information, which is stored on centralized servers and becomes vulnerable to data breaches, unauthorized access, and misuse. Additionally, drivers often face high commission fees and unclear pricing policies, which reduce their earnings and create dissatisfaction. Passengers, on the other hand, experience limited transparency in fare calculation and rating systems, leading to reduced trust in the platform.

Moreover, centralized systems are prone to single points of failure. Any server outage or technical issue can disrupt the entire service, making it unreliable. The absence of transparency and user control further limits accountability in transactions and dispute resolution.

Therefore, there is a need for a decentralized, secure, and transparent ride-sharing system that empowers users, ensures data privacy, reduces dependency on intermediaries, and enhances trust and reliability.

1.3. Objectives:

- To design and develop a decentralized peer-to-peer ride-sharing platform that eliminates the need for a central authority.
- To implement blockchain technology for secure, transparent, and tamper-proof transaction management" into the learning platform for updated and relevant knowledge.
- To utilize the Proof of Authority (PoA) consensus mechanism for faster and cost-efficient transaction validation.
- To enable direct interaction between riders and drivers, reducing dependency on intermediaries.
- To develop an escrow-based payment system that ensures fair and secure

financial transactions between users.

- To build a tamper-proof reputation system that maintains trustworthy and verifiable user ratings.
- To enhance data privacy and security by avoiding centralized storage of sensitive user information.
- To provide a user-friendly interface with login-free access for improved usability and convenience.
- To ensure scalability and efficiency of the system for handling real-time ride requests and transactions.
- To create a foundation for future enhancements, such as AI-based route optimization, dynamic pricing, and token-based payments.

1.4. Scope :

The The scope of the proposed decentralized peer-to-peer ride-sharing system using blockchain technology focuses on developing a secure, transparent, and efficient platform that eliminates the need for centralized intermediaries. The system is designed to enable direct interaction between riders and drivers through a decentralized network, ensuring improved trust, reduced operational costs, and enhanced data privacy. By leveraging a private blockchain with the Proof of Authority (PoA) consensus mechanism, the platform aims to provide fast and reliable transaction processing suitable for real-time ride-booking applications.

The project encompasses the implementation of core functionalities such as ride request generation, driver acceptance, ride completion, and escrow-based payment handling. All transactions are securely recorded on the blockchain, ensuring immutability and transparency. Additionally, a tamper-proof reputation system is included to maintain trust by storing user ratings in a verifiable manner. The system also incorporates login-free access, simplifying user interaction while preserving privacy.

The scope further extends to integrating cryptographic techniques, such as SHA-256 hashing, to ensure data integrity and security. The platform is designed with scalability in mind, allowing future enhancements such as token-based payment systems, AI-driven route optimization, and dynamic pricing mechanisms.

However, the current implementation is limited to a prototype-level system with simulated transactions and a private blockchain environment. Real-time payment gateway integration and large-scale deployment are considered outside the immediate scope but remain potential areas for future development

1.5. Applications :

The proposed decentralized peer-to-peer ride-sharing system using blockchain technology has a wide range of practical applications across various domains of transportation and mobility services. One of the primary applications is in urban transportation, where the system can provide a reliable and cost-effective alternative to traditional ride-sharing platforms. By eliminating intermediaries, it reduces operational costs and offers fair pricing for both riders and drivers.

Another significant application is in carpooling services, where individuals traveling in the same direction can share rides securely and efficiently. The decentralized nature ensures transparency in fare distribution and enhances trust among participants. The system can also be applied in corporate transportation, enabling organizations to manage employee commuting in a secure and optimized manner while maintaining data privacy.

In addition, the platform can support smart city initiatives, where decentralized mobility solutions are essential for efficient traffic management and sustainable urban development. The integration of blockchain ensures secure data sharing between stakeholders without compromising user privacy.

The system also has potential applications in logistics and delivery services, where peer-to-peer coordination can optimize delivery routes and reduce costs. Furthermore, it can be extended to autonomous vehicle networks, where vehicles interact directly through blockchain for ride allocation and payment processing.

Overall, the proposed system provides a flexible and scalable framework that can be adapted to various transportation ecosystems, promoting transparency, efficiency, and user empowerment in modern mobility solutions.

1.6. Limitations :

- The system relies on a private blockchain, which may limit decentralization compared to public blockchain networks.
- It has limited scalability when handling a large number of users and transactions simultaneously.
- The implementation currently uses simulated escrow payments instead of real-time financial integration.
- Initial setup and deployment of blockchain infrastructure require technical complexity and expertise.
- The Proof of Authority (PoA) mechanism depends on trusted nodes, which may

introduce partial centralization.

- The system may experience latency in transaction validation during high network activity.
- Lack of widespread adoption of blockchain technology may affect user acceptance and usability.

CHAPTER 2

LITERATURE SURVEY

[1] S. Ali and F. Bao (2019) – Blockchain-Based Smart Mobility System for Urban Transport.

This study presents a blockchain-based smart mobility framework aimed at improving urban transportation systems through decentralized coordination. The authors propose a system where vehicles, users, and infrastructure interact securely using blockchain technology to enable shared mobility services. The framework emphasizes transparency, traceability, and accountability in ride-sharing operations by storing transactions in an immutable distributed ledger. Smart contracts are utilized to automate ride allocation, fare calculation, and payment settlement, reducing the need for centralized control.

A key contribution of this work is the integration of mobility data with blockchain to optimize ride-sharing efficiency and reduce traffic congestion. The system also supports dynamic ride matching based on user preferences and location data. However, the study highlights challenges such as scalability issues and increased computational overhead due to blockchain operations.

The research demonstrates that blockchain can significantly enhance trust among users by ensuring data integrity and preventing unauthorized modifications. It also provides a foundation for integrating future technologies like IoT and AI into transportation systems. This work is highly relevant to the proposed project as it establishes the importance of decentralization in mobility solutions and highlights the need for efficient consensus mechanisms to support real-time ride-sharing applications.

[2] T. Nguyen and K. Kim (2020) – Trustful Decentralized Transportation Platform Using Blockchain Technology.

Muhammad This research proposes a decentralized transportation platform that leverages blockchain technology to eliminate intermediaries in ride-sharing services. The system enables direct peer-to-peer communication between drivers and riders, ensuring secure and transparent transactions. The authors focus on enhancing user trust by implementing immutable records of rides and ratings on the blockchain.

The platform utilizes smart contracts to automate key processes such as ride booking, payment handling, and dispute resolution. One of the major advantages highlighted in this study is the improved data privacy, as users maintain control over their personal information without relying on a centralized authority. Additionally, the system ensures fair pricing and prevents manipulation of ratings or transaction

data.

Despite its advantages, the study identifies limitations related to transaction speed and scalability, particularly when implemented on public blockchain networks. The authors suggest that alternative consensus mechanisms could improve system efficiency.

This work contributes significantly to the proposed project by demonstrating how blockchain can enhance transparency and trust in ride-sharing systems. It also reinforces the importance of decentralized identity and reputation management, which are key components of the developed system.

[3] P. K. Sharma and J. Lee (2021) – Secure and Efficient Carpooling Using Ethereum Blockchain.

This paper introduces a secure carpooling system built on the Ethereum blockchain, focusing on automating ride-sharing processes using smart contracts. The proposed system enables users to request and offer rides while ensuring secure and transparent transactions through blockchain technology. Smart contracts are used to manage ride agreements, fare calculations, and payment transfers without the involvement of intermediaries.

A notable feature of this system is its ability to prevent fraud by maintaining immutable records of all transactions. The use of Ethereum ensures decentralized execution of contracts, providing a high level of trust among participants. The study also discusses the use of cryptographic techniques to secure user data and ensure authentication.

However, the authors highlight challenges such as high transaction fees and slower processing times associated with public blockchains like Ethereum. These limitations make it less suitable for real-time applications without optimization.

This research is relevant to the proposed system as it demonstrates the effectiveness of smart contracts in automating ride-sharing operations. It also emphasizes the need for alternative blockchain solutions, such as private networks with efficient consensus mechanisms, to overcome scalability and cost-related issues

[4] L. Wang and C. Xu (2021) – Privacy-Preserving Ride-Sharing Framework Using Blockchain.

Pinelopi This study focuses on enhancing user privacy in ride-sharing systems through blockchain technology. The authors propose a privacy-preserving framework that uses advanced cryptographic techniques, including anonymization and zero-knowledge proofs, to protect user identities while maintaining data integrity.

The system ensures that sensitive information such as user identity and location

data is not exposed to unauthorized entities. Blockchain is used to store transaction records securely, while cryptographic methods ensure that only authorized parties can access relevant data. This approach significantly reduces the risk of data breaches and unauthorized tracking.

The research also discusses the trade-off between privacy and system performance, as implementing advanced cryptographic techniques can increase computational complexity. Despite this, the framework successfully demonstrates how privacy can be maintained without compromising transparency.

This work is highly relevant to the proposed project as it highlights the importance of data privacy in decentralized systems. It provides valuable insights into integrating secure identity management mechanisms, which can enhance user trust and system reliability

[5] H. Zhang and Y. Chen (2022) – Decentralized Ride-Sharing Using Blockchain and Edge Computing.

This paper proposes a hybrid approach that combines blockchain technology with edge computing to improve the performance of ride-sharing systems. The integration of edge computing reduces latency by processing data closer to the source, while blockchain ensures secure and transparent transaction management.

The system enables efficient ride matching and real-time decision-making by distributing computational tasks across edge nodes. This approach significantly improves system responsiveness compared to traditional blockchain-based systems. Additionally, the use of blockchain ensures data integrity and prevents tampering.

One of the key contributions of this work is the reduction of network congestion and improved scalability. However, the study also identifies challenges related to system complexity and the need for efficient coordination between edge nodes and the blockchain network.

This research is relevant to the proposed system as it demonstrates how performance limitations of blockchain can be addressed using complementary technologies. It highlights the importance of optimizing system architecture to achieve scalability and efficiency.

[6] E. B. Tirkolaee and M. A. Esfahani (2023) – Blockchain for Ridesharing: A Systematic Literature Review

Katja This paper provides a comprehensive review of blockchain-based ride-sharing systems, analyzing various approaches and technologies used in the field. The authors compare different consensus mechanisms, including Proof of Work (PoW), Proof of Stake (PoS), and Proof of Authority (PoA), based on their scalability, cost, and efficiency.

The study highlights the advantages of blockchain in improving transparency, security, and trust in ride-sharing platforms. It also discusses common challenges such as high transaction costs, low throughput, and limited user adoption.

One of the key findings is that private blockchain networks with lightweight consensus mechanisms, such as PoA, are more suitable for real-time applications. The paper also emphasizes the importance of integrating additional technologies like AI and IoT for enhanced functionality.

This literature survey is directly relevant to the proposed project as it supports the selection of PoA as the consensus mechanism. It provides a strong theoretical foundation for designing an efficient and scalable decentralized ride-sharing system

[7] A. Kumar and S. R. Singh (2023) – Secure Decentralized Ride-Sharing System Using Smart Contracts.

Jiawei This research proposes a decentralized ride-sharing framework that uses smart contracts to automate ride booking and payment processes. The system ensures transparency and eliminates intermediaries by executing all operations on a blockchain network.

The authors highlight the importance of trust in ride-sharing systems and demonstrate how blockchain can provide a secure environment for transactions. The system also includes a reputation mechanism to evaluate user behavior and maintain service quality.

Despite its advantages, the study identifies limitations related to scalability and transaction speed. The authors suggest optimizing smart contract execution and exploring alternative consensus mechanisms to improve performance.

This work contributes to the proposed project by demonstrating the practical implementation of blockchain-based ride-sharing systems. It reinforces the importance of automation and transparency in building reliable platforms

[8] J. Wessly and R. Durga (2024) – Blockchain Ethereum-Based Secure Ride-Sharing Framework.

Muhusina This paper introduces a secure ride-sharing framework based on Ethereum blockchain. The system incorporates role-based access control and cryptographic techniques to ensure secure communication between users.

The framework focuses on enhancing trust and transparency by maintaining immutable ride records. It also supports decentralized identity management, allowing users to interact securely without revealing sensitive information.

However, the study highlights challenges such as high gas fees and limited scalability associated with Ethereum. The authors suggest exploring alternative

blockchain platforms for better performance.

This research is relevant as it emphasizes the importance of security and access control in decentralized systems, which are key components of the proposed project.

[9] Wanxin Li et al. (2025) – Bidirectional Authentication for Safe Ridesharing Using Blockchain.

E. This study focuses on improving security in ride-sharing systems through bidirectional authentication. The proposed system ensures that both riders and drivers are verified before initiating a transaction, reducing the risk of fraud.

Blockchain is used to store authentication data securely, ensuring transparency and preventing unauthorized access. The system also integrates cryptographic techniques to protect user information.

The research highlights the importance of secure authentication mechanisms in decentralized systems. It also discusses challenges related to implementation complexity and system performance.

This work is relevant as it provides insights into enhancing security and trust in ride-sharing platforms.

[10] Aqsa Ashraf Makhdomi and Iqra Altaf Gillani (2025) – Collaborative Fair Route Planning in Decentralized Platforms.

This paper explores fair route planning in decentralized ride-sharing systems using game theory. The proposed approach ensures equitable distribution of resources among users, improving system efficiency and user satisfaction.

The study highlights the importance of fairness in decentralized systems and demonstrates how advanced algorithms can optimize ride allocation. It also discusses challenges related to computational complexity and scalability.

This research is relevant as it provides insights into improving efficiency and fairness in ride-sharing systems, which can be incorporated into future enhancements of the proposed project.

CHAPTER 3

ANALYSIS & DESIGN

System analysis and design is a crucial phase in the development of the proposed decentralized peer-to-peer ride-sharing system using blockchain technology. This phase focuses on understanding the system requirements, defining the architecture, and designing an efficient structure that ensures scalability, security, and performance. The objective is to create a robust system that enables seamless interaction between riders and drivers while maintaining transparency and data integrity.

The analysis begins with identifying the limitations of existing centralized ride-sharing platforms, such as lack of transparency, high operational costs, and data privacy concerns. Based on these challenges, the proposed system is designed to operate on a decentralized blockchain network, eliminating the need for intermediaries. The system requirements include secure transaction handling, efficient ride matching, reliable reputation management, and real-time processing capabilities.

The design phase involves structuring the system into multiple components, including the user interface, backend services, blockchain ledger, and reputation management module. The architecture follows a modular approach, where each component performs a specific function, ensuring flexibility and ease of maintenance. The blockchain layer is responsible for recording all transactions in an immutable manner, while the backend handles business logic and communication between users.

Data flow within the system is carefully designed to ensure smooth execution of ride requests, driver acceptance, ride completion, and payment processing. Cryptographic techniques such as SHA-256 hashing are incorporated to maintain data security and prevent unauthorized modifications.

Overall, the system analysis and design provide a comprehensive blueprint for developing a secure, efficient, and scalable decentralized ride-sharing platform that addresses the limitations of traditional systems while ensuring reliability and user trust.

1.1. Existing System :

The existing ride-sharing systems, such as Uber, Lyft, and Ola, operate on centralized architectures where a single organization manages all aspects of the platform, including user data, ride allocation, pricing, and payment processing. These platforms have significantly improved urban transportation by providing convenient and on-demand ride services. However, despite their widespread

adoption and success, they suffer from several inherent limitations due to their centralized nature.

One of the major drawbacks of existing systems is the lack of transparency. Pricing algorithms, surge charges, and commission structures are controlled by the central authority, often without clear explanation to users or drivers. This results in dissatisfaction, particularly among drivers who face high commission fees and sudden changes in policies. Additionally, passengers have limited visibility into how fares are calculated, which reduces trust in the system.

Data privacy and security are also critical concerns. Centralized platforms store large volumes of sensitive user information, including personal details, ride history, and payment data, making them attractive targets for cyberattacks. Any breach in the system can lead to significant data loss and misuse of personal information.

Another limitation is the dependency on a central server, which creates a single point of failure. System outages or technical issues can disrupt services entirely, affecting both riders and drivers. Furthermore, users have limited control over their data and transactions, as all operations are governed by the platform provider.

Overall, while existing ride-sharing systems offer convenience, they lack transparency, security, and user autonomy. These limitations highlight the need for a decentralized solution that can provide greater control, trust, and efficiency in ride-sharing services

1.2. Disadvantages of Existing System :

Despite the widespread success of centralized ride-sharing platforms such as Uber, Lyft, and Ola, they exhibit several significant disadvantages that impact users, drivers, and overall system reliability. These limitations arise primarily due to their centralized architecture and dependency on a single controlling authority.

One of the major disadvantages is lack of transparency. Users and drivers are not fully aware of how fares are calculated, how ratings are managed, or how decisions are made by the platform. Surge pricing and commission deductions are controlled internally, often without clear justification, leading to distrust among users.

Another critical issue is poor data privacy and security. Centralized systems store large amounts of sensitive personal and financial data on centralized servers, making them vulnerable to cyberattacks, data breaches, and unauthorized access. A single security failure can expose millions of users’ information.

The high commission structure imposed on drivers is another major drawback. A

significant portion of the fare is deducted by the platform, reducing driver earnings and leading to dissatisfaction among service providers. Similarly, passengers often face increased costs during peak demand periods.

Centralized systems also suffer from a single point of failure, meaning that if the central server experiences downtime or technical issues, the entire service becomes unavailable. This affects both riders and drivers, causing disruption in transportation services.

Additionally, users have limited control over their data and transactions, as all operations are governed by the service provider. This lack of autonomy reduces user empowerment and trust in the system.

Overall, these disadvantages highlight the need for a decentralized, secure, and transparent ride-sharing system that eliminates intermediaries and provides better control, fairness, and reliability for all participants.

1.3. Proposed System :

The proposed system introduces a decentralized peer-to-peer ride-sharing platform using blockchain technology to overcome the limitations of existing centralized systems. The system is designed to eliminate intermediaries and enable direct communication between riders and drivers, ensuring greater transparency, security, and efficiency in ride-sharing operations.

At the core of the proposed system is a private blockchain network utilizing the Proof of Authority (PoA) consensus mechanism, which ensures fast, reliable, and low-cost transaction validation. Unlike traditional public blockchains, PoA relies on trusted validator nodes, making it suitable for real-time applications such as ride booking and payment processing.

The system incorporates a multi-stage ride lifecycle model, where each action—ride request, driver acceptance, ride completion, and payment settlement—is recorded as a separate blockchain transaction. This ensures complete traceability and immutability of all ride activities. An escrow-based payment system is integrated to securely hold funds until the ride is successfully completed, ensuring fairness for both riders and drivers.

A key feature of the proposed system is the decentralized reputation mechanism, where user ratings are stored on the blockchain in an immutable format. This prevents manipulation of reviews and builds trust among participants. Additionally, login-free ride booking enhances user convenience while maintaining privacy

through secure transaction identifiers.

The system also uses cryptographic hashing (SHA-256) to maintain data integrity and ensure that any modification to stored information can be easily detected. All components are designed using a modular architecture, allowing scalability and future enhancements such as AI-based route optimization, dynamic pricing, and token-based reward systems.

Overall, the proposed system provides a secure, transparent, and efficient alternative to traditional ride-sharing platforms by leveraging blockchain technology to ensure trust, fairness, and decentralized control.

1.4. Advantages of Proposed System :

The proposed decentralized peer-to-peer ride-sharing system using blockchain technology offers several significant advantages over traditional centralized ride-sharing platforms. These advantages primarily stem from the use of decentralization, cryptographic security, and transparent transaction management.

One of the key advantages is enhanced transparency. Since all ride transactions, including ride requests, acceptance, completion, and payments, are recorded on the blockchain, every action is traceable and verifiable. This eliminates hidden charges and ensures fair dealings between riders and drivers.

Another major benefit is improved data security and privacy. Unlike centralized systems that store user data in a single database, the proposed system distributes data across a blockchain network. This reduces the risk of data breaches, unauthorized access, and misuse of sensitive information.

The system also ensures fair and secure payments through an escrow mechanism. Funds are securely locked until the ride is completed, ensuring that drivers receive payment only after successful service delivery, while riders are protected from fraudulent activities.

A significant advantage is the tamper-proof reputation system, where user ratings are stored on the blockchain. This prevents manipulation of reviews and ensures that reputation scores are accurate and trustworthy.

The use of the Proof of Authority (PoA) consensus mechanism enables faster transaction processing with lower computational overhead, making the system suitable for real-time ride-sharing applications.

Additionally, the system reduces dependency on intermediaries, thereby

eliminating high commission fees and increasing earnings for drivers while reducing costs for passengers.

Finally, the modular and scalable design of the system allows for future enhancements such as AI-based route optimization, dynamic pricing, and token-based reward systems.

Overall, the proposed system provides a secure, efficient, and user-centric ride-sharing solution that improves trust, reduces costs, and enhances overall system performance.

1.5. Modules :

User Management Module:

The User Management Module is a fundamental component of the decentralized peer-to-peer ride-sharing system. It is responsible for handling the interactions of riders and drivers within the platform while ensuring secure and efficient access to system functionalities. Unlike traditional systems that require mandatory registration and login credentials, this module supports a **login-free or minimal identity-based access mechanism**, enhancing user convenience while maintaining privacy.

In this module, users (riders and drivers) are identified using unique blockchain-based identifiers instead of storing sensitive personal information in a centralized database. This approach reduces privacy risks and eliminates dependency on traditional authentication systems. Riders can request rides, while drivers can view available ride requests and accept them in real time.

The module also manages basic profile information such as user reputation scores, ride history references, and activity logs, which are stored in a decentralized manner through blockchain transactions. This ensures that user-related data remains tamper-proof and verifiable.

Security is enforced using cryptographic techniques, ensuring that each user action is authenticated before being recorded on the blockchain. The module also integrates with the reputation system, where user behavior is continuously evaluated based on ride completion and feedback.

Overall, the User Management Module ensures seamless interaction between system participants while maintaining decentralization, privacy, and security. It forms the foundation for all other modules by enabling controlled access and reliable user identification in the blockchain-based ride-sharing ecosystem.

Ride Management Module:

The Ride Management Module is the core functional component of the decentralized ride-sharing system. It is responsible for handling the complete lifecycle of a ride, including ride request creation, driver acceptance, ride execution, and completion. This module ensures that every stage of the ride is properly recorded and managed using blockchain transactions.

When a rider initiates a ride request, the module generates a transaction labeled as **ESCROW_PENDING**, which is broadcast to the network. This request becomes visible to available drivers, who can then accept the ride through the system interface. Once a driver accepts the request, a new transaction is created, transitioning the ride status to **RIDE_ACCEPTED_IN_PROGRESS**.

During the ride execution phase, the system continuously tracks the ride state without modifying previous records, ensuring immutability. Upon successful completion of the ride, the final transaction **ESCROW_RELEASED_COMPLETED** is generated, marking the end of the ride lifecycle.

This module ensures transparency by maintaining a complete and immutable history of all ride activities. Each transaction is linked using cryptographic hashes, ensuring that any modification attempt can be easily detected. The module also interacts with the escrow system to ensure secure payment handling, where funds are released only after ride completion.

Additionally, the Ride Management Module plays a key role in integrating with the reputation system by triggering rating updates after ride completion. Overall, this module ensures secure, transparent, and efficient management of ride operations in a decentralized environment.

Blockchain and Consensus Module:

The Blockchain and Consensus Module forms the backbone of the decentralized ride-sharing system. It is responsible for maintaining the distributed ledger, validating transactions, and ensuring data integrity across the network. This module ensures that all ride-related activities are securely recorded in an immutable and verifiable manner.

The system uses a **private blockchain architecture with Proof of Authority (PoA) consensus mechanism**, which is suitable for high-speed and low-cost transaction processing. In PoA, trusted validator nodes are responsible for verifying and adding new blocks to the blockchain. This significantly reduces computational overhead compared to Proof of Work systems.

Each transaction in the system is grouped into blocks, and each block contains a

cryptographic hash of the previous block, ensuring chain integrity. The module uses **SHA-256 hashing algorithm** to generate unique block identifiers, making the ledger tamper-proof.

This module also ensures synchronization across all nodes in the network. Whenever a new block is added, it is propagated to all participants, ensuring consistency of the ledger. Any attempt to modify past data results in hash mismatch, making tampering easily detectable.

The Blockchain and Consensus Module ensures trust, transparency, and immutability in the system. It eliminates the need for centralized authorities while maintaining high performance suitable for real-time ride-sharing applications.

Escrow and Payment Module:

The Escrow and Payment Module is responsible for handling secure financial transactions within the decentralized ride-sharing system. This module ensures that payments are managed fairly between riders and drivers without requiring a central authority.

When a ride is initiated, the fare amount is conceptually placed into an **escrow state (ESCROW_PENDING)**. This means that the payment is temporarily held within the system until the ride is successfully completed. Once the ride reaches the **ESCROW_RELEASED_COMPLETED** state, the payment is released to the driver automatically.

This mechanism ensures fairness and prevents fraud, as drivers are guaranteed payment only after completing the service, while riders are protected from incomplete or unsatisfactory rides.

The module works closely with the blockchain layer, where every payment-related transaction is recorded immutably. This ensures transparency and prevents manipulation of financial records.

Although the current implementation may simulate payments, the architecture is designed to support future integration with real payment gateways and token-based systems. This will further enhance the system’s usability in real-world applications.

Overall, the Escrow and Payment Module ensures secure, transparent, and reliable financial transactions in a decentralized environment, improving trust between users and drivers.

Reputation and Analytics Module:

The Reputation and Analytics Module is designed to maintain trust and evaluate user behavior within the decentralized ride-sharing system. It plays a crucial role in ensuring quality service and accountability among riders and drivers.

After each ride is completed, users provide ratings, which are stored as part of the final blockchain transaction. These ratings are used to update the **reputation score**, which is maintained in a decentralized reputation ledger. The system ensures that reputation data is immutable and cannot be manipulated.

The module uses a weighted algorithm where ratings and fare values influence the final reputation score. Higher-value rides contribute more significantly to reputation changes, ensuring a balanced evaluation system.

In addition to reputation tracking, this module also provides analytical insights into system performance, such as ride frequency, user activity, and transaction patterns. These analytics help improve system efficiency and future decision-making.

The Reputation and Analytics Module ensures fairness, transparency, and trustworthiness in the system. It also supports long-term system growth by enabling data-driven improvements and enhancing user reliability within the decentralized ride-sharing ecosystem.

1.6. Algorithms:

SHA-256 Hashing Algorithm :

The "SHA-256 hashing algorithm" is a widely used cryptographic technique that plays a crucial role in securing user authentication within the proposed system. It is used to convert user passwords into a fixed-length encrypted format, ensuring that sensitive information is not stored in plain text form. The algorithm operates bytaking an input message and processing it through multiple rounds of mathematical transformations, including bitwise operations, modular additions, and compression functions, to generate a unique 256-bit hash value. This process ensures that even a small change in input produces a completely different hash output, enhancing security against brute-force and dictionary attacks [2]. In the context of this system, SHA-256 is implemented within the "secure authentication module" to protect user credentials during registration and login processes. When a user enters a password, it is hashed using SHA-256, and the resulting value is stored in the database. During login, the entered password is hashed again and compared with the stored hash to verify authenticity [5]. This ensures that actual

passwords are never exposed or transmitted directly. Additionally, the algorithm supports "data integrity", "secure authentication", "password protection", and "encryption-based security", making it highly reliable for cybersecurity applications. Its deterministic and irreversible nature makes it computationally infeasible to retrieve the original password from the hash.

Proof of Authority (PoA) Consensus Algorithm:

The Proof of Authority (PoA) consensus algorithm is used as the primary validation mechanism in the proposed blockchain-based ride-sharing system. It is a lightweight and efficient consensus model designed for private blockchain networks where participants are known and trusted. Unlike Proof of Work (PoW), which requires high computational power, PoA relies on a set of pre-approved validator nodes (authorities) to validate transactions and generate new blocks.

In this system, the authority node is responsible for verifying ride-related transactions such as ride requests, driver acceptance, ride completion, and escrow settlement. Once a transaction is validated, it is grouped into a block and added to the blockchain. This ensures that only legitimate and verified transactions are recorded, maintaining system integrity.

PoA significantly improves system performance by reducing the time required for block generation. Since there is no need for complex mathematical puzzle solving, transactions are processed almost instantly. This makes PoA highly suitable for real-time applications like ride-sharing, where quick response time is essential.

Another advantage of PoA is its low energy consumption. Unlike PoW-based systems, it does not require intensive computational resources, making it more efficient and environmentally friendly.

However, PoA depends on trusted validators, which introduces partial centralization. In the proposed system, this is controlled by limiting authority nodes to verified system administrators or trusted entities, ensuring a balance between decentralization and performance.

PoA also enhances scalability, allowing the system to handle a large number of ride requests efficiently. It ensures that all transactions are validated consistently and recorded in an immutable ledger.

Overall, the Proof of Authority algorithm plays a crucial role in ensuring fast, secure, and scalable transaction processing in the decentralized ride-sharing system.

Multi-Transaction Ride Lifecycle Algorithm:

The Multi-Transaction Ride Lifecycle Algorithm is designed to manage the complete journey of a ride in a decentralized and structured manner. Instead of updating a single database record, the system records each stage of the ride as a separate blockchain transaction. This approach ensures immutability, traceability, and transparency.

The lifecycle begins when a rider requests a ride. This action generates the first transaction labeled **ESCROW_PENDING**, which represents a ride request waiting for driver acceptance. This transaction is broadcast to the network and stored in the pending pool.

When a driver accepts the ride, a second transaction is created, transitioning the ride state to **RIDE_ACCEPTED_IN_PROGRESS**. This transaction links back to the original request using a transaction identifier, ensuring continuity of the ride history.

Finally, when the ride is completed, a third transaction is generated with the status **ESCROW_RELEASED_COMPLETED**. This marks the end of the ride lifecycle and triggers the payment release and reputation update process.

Each transaction is cryptographically linked to the previous one, forming a secure chain of ride events. This ensures that every action is permanently recorded and cannot be altered without breaking the blockchain integrity.

The algorithm improves auditability, as every ride can be traced step-by-step from request to completion. It also enhances system reliability by eliminating the need for mutable records.

This structured lifecycle approach is particularly useful in decentralized systems where trust, transparency, and accountability are critical. It ensures that all ride activities are verifiable and tamper-proof, supporting the core objectives of the proposed system.

1.7. System Architecture :

The The Multi-Transaction Ride Lifecycle Algorithm is designed to manage the complete journey of a ride in a decentralized and structured manner. Instead of updating a single database record, the system records each stage of the ride as a separate blockchain transaction. This approach ensures immutability, traceability, and transparency.

The lifecycle begins when a rider requests a ride. This action generates the first transaction labeled **ESCROW_PENDING**, which represents a ride request waiting for driver acceptance. This transaction is broadcast to the network and stored in the pending pool.

When a driver accepts the ride, a second transaction is created, transitioning the ride state to **RIDE_ACCEPTED_IN_PROGRESS**. This transaction links back to the original request using a transaction identifier, ensuring continuity of the ride history.

Finally, when the ride is completed, a third transaction is generated with the status **ESCROW_RELEASED_COMPLETED**. This marks the end of the ride lifecycle and triggers the payment release and reputation update process.

Each transaction is cryptographically linked to the previous one, forming a secure chain of ride events. This ensures that every action is permanently recorded and cannot be altered without breaking the blockchain integrity.

The algorithm improves auditability, as every ride can be traced step-by-step from request to completion. It also enhances system reliability by eliminating the need for mutable records.

This structured lifecycle approach is particularly useful in decentralized systems where trust, transparency, and accountability are critical. It ensures that all ride activities are verifiable and tamper-proof, supporting the core objectives of the proposed system.

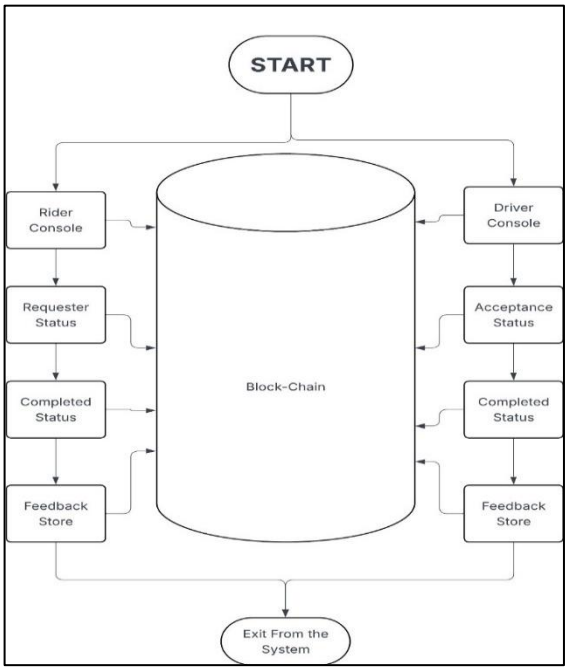

Figure 3.7 : System Architecture

1.8. UML Diagrams :

1.8.1. Use Case Diagram :

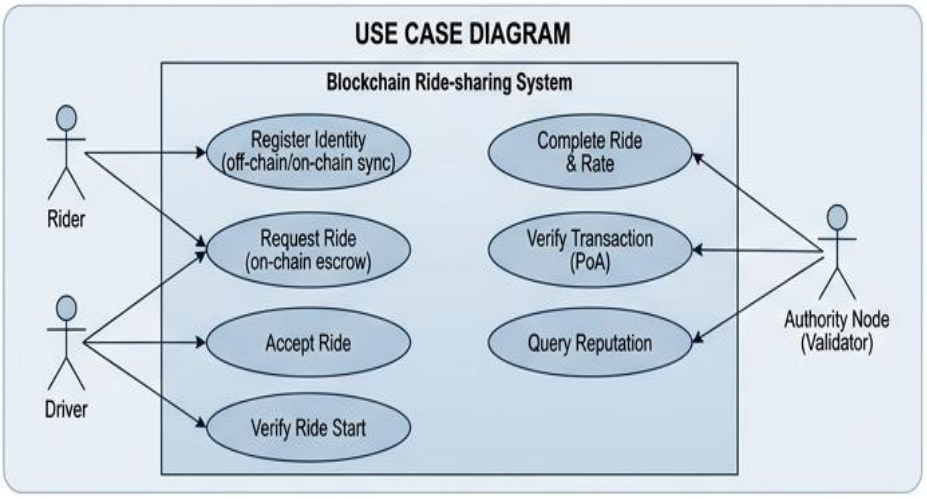

Figure 3.8.1 : Use case Diagram

The Use Case Diagram of the decentralized peer-to-peer ride-sharing system represents the functional interactions between the primary actors and the system. The main actors in this system are Rider, Driver, and Admin, each having specific roles and responsibilities. This diagram helps in understanding how different users interact with the system to achieve ride-sharing operations in a decentralized environment.

The Rider is responsible for initiating ride requests, making payments through the escrow system, viewing ride status, and providing ratings after ride completion. The Driver interacts with the system by accepting ride requests, completing trips, and also rating riders. The Admin plays a crucial role in maintaining system integrity by validating blockchain transactions and monitoring the overall network performance.

The key use cases include **Request Ride, Accept Ride, Complete Ride, Escrow Payment Processing, Rate User, View Reputation, and Validate Transactions**. When a rider requests a ride, the system generates a blockchain transaction and broadcasts it to available drivers. Drivers can accept the ride request, which updates the ride status in the blockchain ledger. Once the ride is completed, the escrow mechanism releases the payment to the driver, ensuring secure financial settlement.

The reputation system is another important use case, where both riders and drivers are rated based on their behavior and service quality. These ratings are stored immutably on the blockchain, preventing manipulation or fraudulent changes.

The Admin ensures that all transactions are valid and that the Proof of Authority (PoA) consensus mechanism functions correctly. This helps maintain trust and reliability in the decentralized system.

Overall, the Use Case Diagram clearly illustrates the interaction between users and system functionalities, highlighting the decentralized nature of the ride-sharing platform and ensuring transparency, security, and trust in all operations.

1.8.2. Class Diagram:

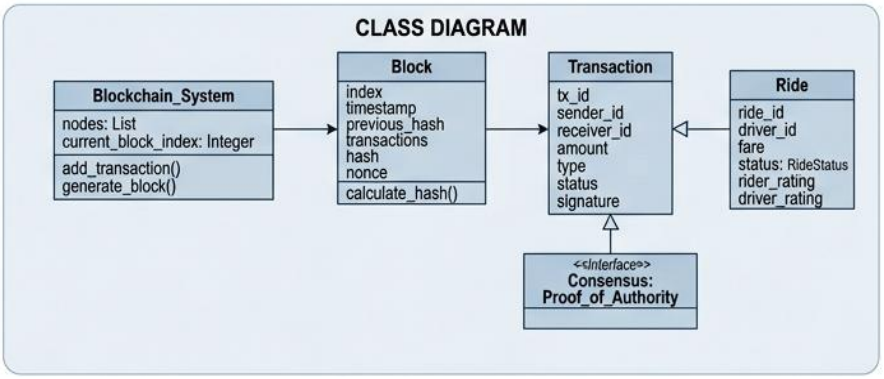

Figure 3.8.2: Class Diagram

The Class Diagram represents the structural design of the decentralized ride-sharing system by defining its key classes, attributes, and relationships. It provides a blueprint of how different components interact and are organized within the system architecture.

The main classes in the system include User, Rider, Driver, Ride, Transaction, Blockchain, and Reputation System. The User class acts as a base class, containing common attributes such as user ID, name, and reputation score. The Rider and Driver classes inherit from the User class, extending its functionalities with specific methods like requestRide() for riders and acceptRide() for drivers.

The Ride class represents the ride lifecycle and contains attributes such as ride ID, status, fare, and timestamps. It is closely associated with the Transaction class, which records all blockchain-related activities including ride creation, acceptance, and completion. Each ride generates multiple transactions, ensuring a complete and immutable history of events.

The Blockchain class is responsible for managing blocks, validating transactions, and maintaining the distributed ledger. It includes methods such as addBlock() and validateBlock(), which ensure data integrity and consistency across the network.

The Reputation System class maintains user trust scores based on ride ratings

and activity. It updates reputation values after each completed ride using a defined algorithm.

Relationships between classes include inheritance (User → Rider/Driver), associations (Ride linked to Transaction), and dependencies (Blockchain validates transactions).

Overall, the Class Diagram provides a clear structural representation of the system, ensuring modularity, scalability, and maintainability in the decentralized ride-sharing platform.

1.8.3. Sequence Diagram :

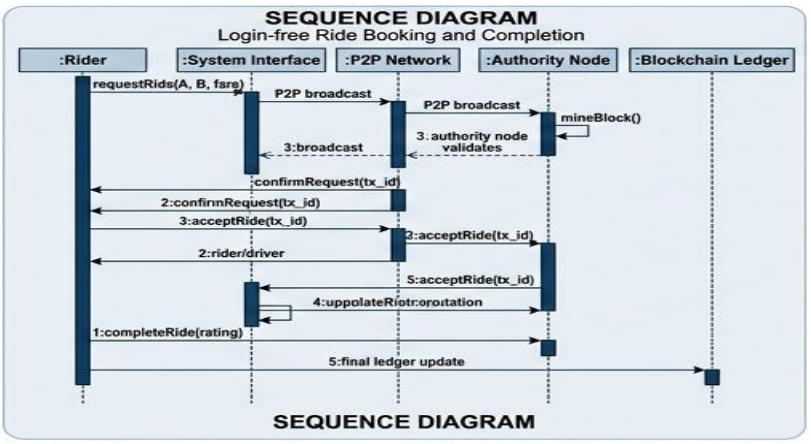

Figure 3.8.3: Sequence Diagram

The Sequence Diagram illustrates the step-by-step interaction between different components of the decentralized ride-sharing system during the execution of a ride. It shows the chronological flow of messages between the Rider, Driver, System, and Blockchain.

The process begins when the Rider sends a ride request to the system. The system then creates a transaction labeled ESCROW_PENDING and forwards it to the blockchain network for validation. Once the transaction is verified under the Proof of Authority (PoA) consensus mechanism, it is added to the blockchain ledger.

Next, the Driver receives the ride request and sends an acceptance response. The system updates the ride status and creates a new transaction linking the driver to the ride. This transaction updates the state to RIDE_ACCEPTED_IN_PROGRESS, ensuring traceability.

During the ride execution phase, no modification is made to previous transactions. Instead, the system maintains state continuity through new linked transactions.

Once the ride is completed, the Rider triggers the ride completion event, and the

system generates the final transaction ESCROW_RELEASED_COMPLETED. This transaction initiates the escrow release process, transferring payment to the driver.

Finally, both rider and driver provide ratings, which are stored on the blockchain and used to update the reputation system.

The Sequence Diagram clearly demonstrates the interaction flow and ensures transparency, immutability, and trust in the decentralized ride-sharing process.

1.8.4. Activity Diagram:

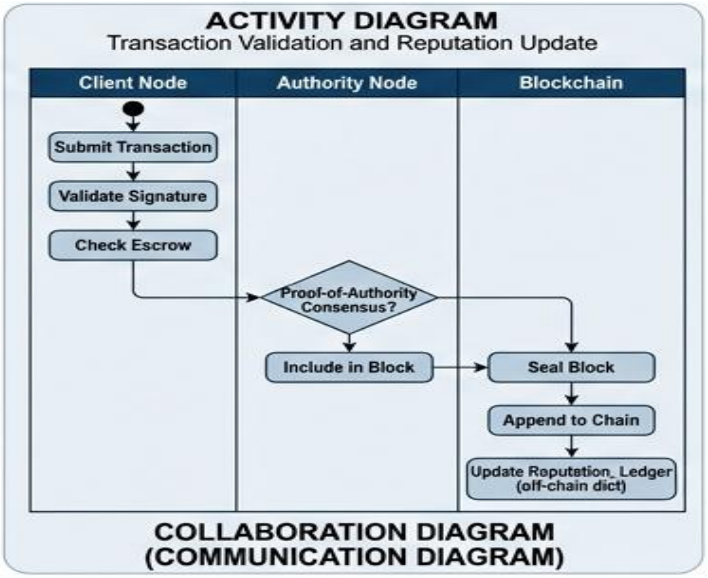

Figure 3.8.4: Activity Diagram

The Activity Diagram represents the workflow of the decentralized ride-sharing system, showing the sequence of operations from ride initiation to completion. It provides a clear visualization of the system's business logic and decision flow.

The process begins when a Rider initiates a ride request. This request is recorded as a blockchain transaction under the ESCROW_PENDING state. The system then broadcasts this request to available drivers.

Drivers evaluate incoming requests and select a suitable ride. Once a Driver accepts the request, the system updates the ride state to RIDE_ACCEPTED_IN_PROGRESS. At this stage, the ride becomes active and is assigned to a specific driver.

During the ride execution phase, the system continuously tracks ride status without altering previous records, ensuring immutability.

After the ride is completed, the Rider confirms completion, triggering the ESCROW_RELEASED_COMPLETED state. This step initiates the payment release process from escrow to the driver.

Following payment settlement, both rider and driver submit ratings. These ratings are processed by the reputation system, which updates user scores based on predefined algorithms.

The activity flow ends after all updates are recorded on the blockchain, ensuring a complete and verifiable audit trail.

This diagram highlights the sequential and conditional flow of operations, ensuring transparency, security, and efficiency in the decentralized ride-sharing system.

1.8.5. Collaboration Diagram :

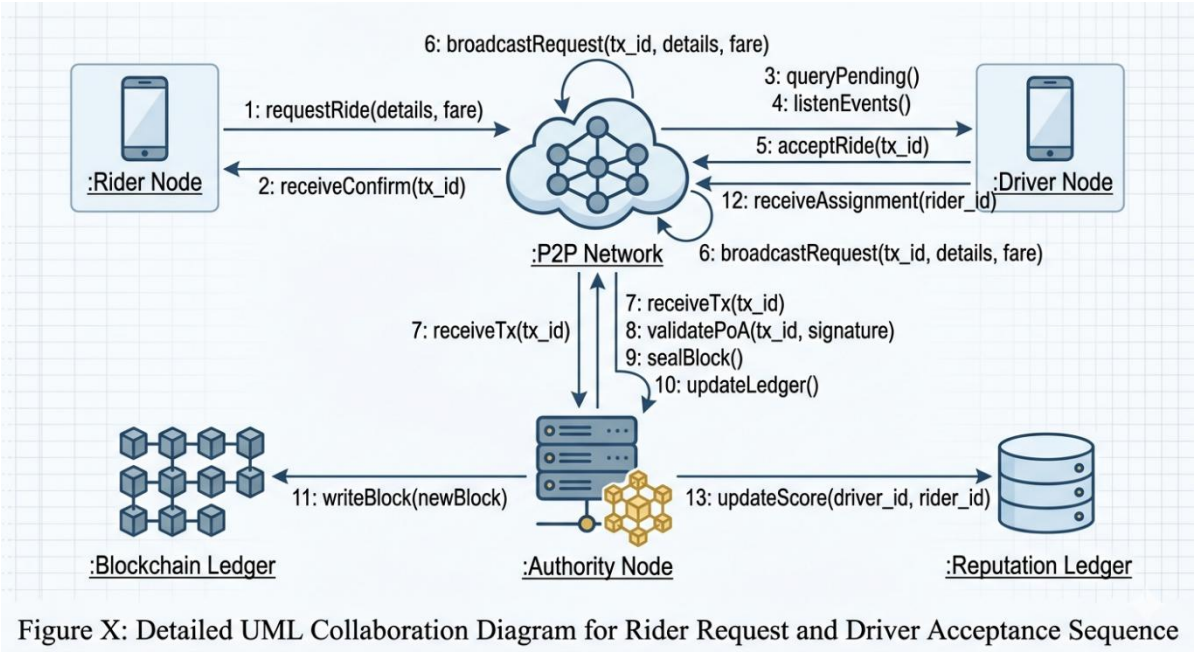

Figure 3.8.5 : Collaboration Diagram

The Collaboration Diagram represents the interaction between system components emphasizing message exchange and relationships among objects in the decentralized ride-sharing system.

The main objects involved are Rider, Driver, System Interface, and Blockchain Network. These components collaborate to execute ride-sharing operations in a decentralized environment.

The process begins when the Rider sends a ride request to the System Interface. The system then forwards this request to the Blockchain Network, where it is recorded as a transaction. This ensures immutability and transparency of ride requests.

The Driver receives notification of available rides and sends an acceptance message back to the system. The system updates the blockchain ledger by creating

a new transaction linking the driver to the ride.

During ride execution, the System continuously interacts with the Blockchain to maintain state consistency. Once the ride is completed, the Rider sends a completion request, which triggers escrow release and final transaction recording.

The Blockchain plays a central role in ensuring data integrity, while the System Interface acts as a communication bridge between users and the blockchain.

This diagram highlights how decentralized components collaborate without a central authority, ensuring transparency, trust, and secure communication between participants.

1.8.6. Deployment Diagram:

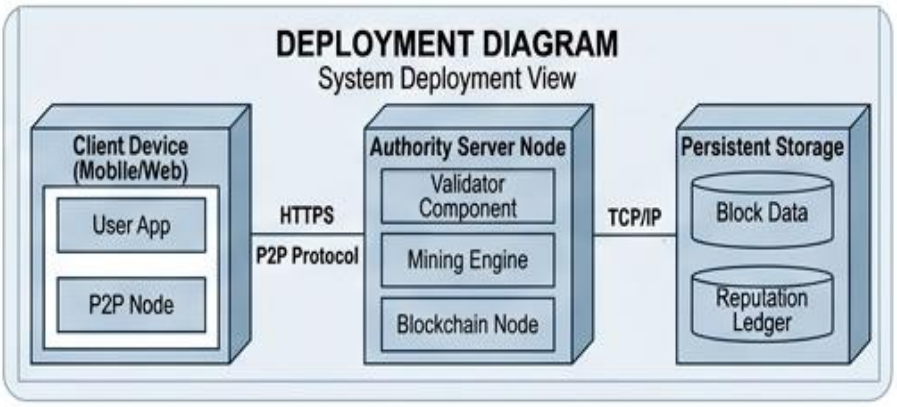

Figure 3.8.6 : Deployment Diagram

The Deployment Diagram represents the physical architecture of the decentralized ride-sharing system and shows how software components are distributed across hardware nodes.

The system consists of three main layers: **User Device Layer, Backend Server Layer, and Blockchain Network Layer**. The User Device Layer includes mobile or web applications used by riders and drivers to interact with the system.

The Backend Server Layer hosts the ride management API, business logic, and blockchain interface engine. It handles ride requests, driver matching, and communication between users and the blockchain network.

The Blockchain Network Layer consists of validator nodes operating under the Proof of Authority (PoA) consensus mechanism. These nodes are responsible for validating transactions and maintaining the distributed ledger.

When a user performs an action such as requesting or completing a ride, the request is sent from the frontend to the backend server. The backend processes the request and forwards it to the blockchain engine, which interacts with validator

nodes to add transactions to the ledger.

This layered architecture ensures separation of concerns, scalability, and fault tolerance. It also improves system performance by distributing tasks efficiently across different components.

Overall, the Deployment Diagram provides a clear understanding of how the decentralized system is physically implemented and deployed in real-world environments.

1.8.7. ER Diagram :

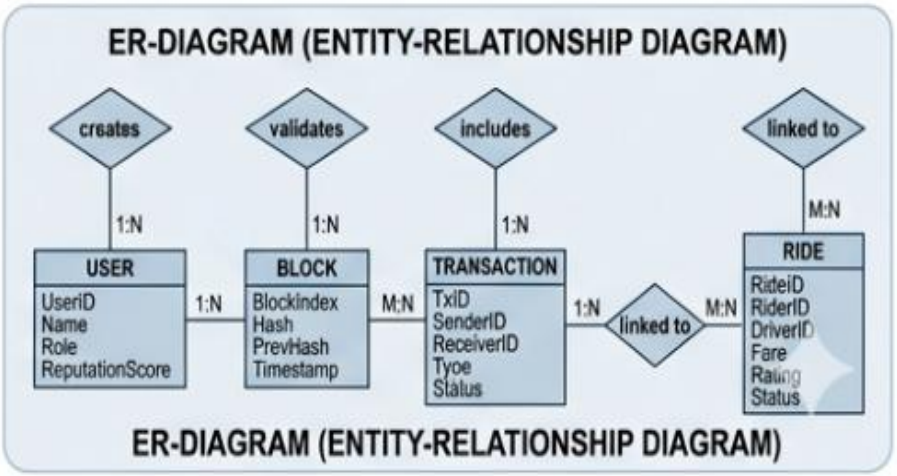

Figure 3.8.7 : ER Diagram

The Entity-Relationship (ER) Diagram represents the logical structure of data in the decentralized ride-sharing system and shows relationships between key entities.

The main entities in the system are **Rider, Driver, Ride, Transaction, and Reputation**. Each entity contains specific attributes that define its properties. For example, the Rider entity includes attributes such as rider ID, name, and reputation score, while the Driver entity includes driver ID, name, and rating.

The Ride entity represents the core operational unit and contains details such as ride ID, fare, status, and timestamps. Each ride is associated with both a rider and a driver, establishing a one-to-many relationship.

The Transaction entity stores blockchain records for each stage of the ride lifecycle, including ride request, acceptance, and completion. This ensures immutability and traceability of all activities.

The Reputation entity maintains user trust scores, which are updated after each completed ride based on ratings received.

Relationships between entities include:

- One Rider can have multiple Rides
- One Driver can handle multiple Rides
- Each Ride generates multiple Transactions
- Each User has one Reputation record

This ER model ensures structured data management, consistency, and efficient retrieval of information in the decentralized system.

CHAPTER 4

IMPLEMENTATION

4.1. Overview of Implementation:

The implementation of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology focuses on developing a secure, transparent, and efficient platform that enables direct interaction between riders and drivers without relying on centralized intermediaries. The system is implemented using a combination of backend development, blockchain simulation, and modular service-based architecture to ensure scalability and reliability.

The core implementation is built around a private blockchain framework that records all ride-related activities such as ride requests, acceptance, completion, and payment settlement. Each action is treated as a transaction and stored in an immutable ledger. The blockchain ensures that all records are tamper-proof and verifiable, providing transparency across the system.

The system follows a multi-transaction ride lifecycle model, where every stage of a ride is recorded separately. This includes states such as ESCROW_PENDING, RIDE_ACCEPTED_IN_PROGRESS, and ESCROW_RELEASED_COMPLETED. These states help in maintaining a clear and traceable history of each ride from initiation to completion.

A key part of the implementation is the escrow-based payment mechanism, which ensures that payments are securely held until the ride is successfully completed. Once the ride is completed, the payment is automatically released to the driver through blockchain validation.

The system also includes a decentralized reputation module, where user ratings are recorded after each ride. These ratings are stored on the blockchain to ensure they cannot be modified or manipulated, thereby maintaining trust between users.

Additionally, cryptographic techniques such as SHA-256 hashing are used to ensure data integrity and secure linking of blockchain blocks. The system is designed using a modular architecture, separating functionalities such as ride management, blockchain operations, and user interaction layers.

The implementation is carried out using Python-based backend services along with simulated blockchain logic, making it suitable for prototype-level deployment

and further extension into real-world applications. Overall, the implementation successfully demonstrates how blockchain technology can be applied to create a secure, transparent, and decentralized ride-sharing ecosystem

4.2. Hardware Requirements:

RAM : 8 GB

Processor : Intel i5 or higher

Storage : 256 GB (SSD preferred)

System Type : 64-bit Operating System

Network : Stable Internet Connection

Device : Laptop / Desktop System

4.3. Software Requirements :

Operating System : Windows 10 / Linux / macOS

Programming Language : Python

Frontend Framework : Streamlit

Cybersecurity Tools : Metasploit, Wireshark, Kali Linux

Data Source : Real-time Simulated Cybersecurity Scenarios

Dev Tools : VS Code / PyCharm

4.4. Functional & Non-Functional Requirements :

- The system shall allow riders to request a ride by submitting location and destination details through the platform.
- The system shall enable drivers to view and accept available ride requests in real time based on proximity and availability.
- The system shall record every ride request, acceptance, and completion as a separate blockchain transaction.
- The system must ensure "high security" by implementing encryption, secure authentication, and data protection mechanisms. It should prevent unauthorized access and maintain user data confidentiality.
- The system should provide "high performance" with minimal response time during user interactions and module execution. It should efficiently handle multiple user requests without system delays.
- The system must be "scalable" to support increasing numbers of users and

additional modules in the future. It should allow easy integration of new features and updates.

- The system should maintain "usability and reliability" through a user-friendly interface and consistent performance.
- The system shall implement a multi-stage ride lifecycle, including states such as ESCROW_PENDING, RIDE_ACCEPTED_IN_PROGRESS, and ESCROW_RELEASED_COMPLETED
- The system shall provide an escrow-based payment mechanism that holds funds until the ride is successfully completed.
- The system shall automatically release payment to the driver once ride completion is verified on the blockchain.
- The system shall allow both riders and drivers to rate each other after ride completion.
- The system shall maintain a decentralized reputation system where ratings are stored immutably on the blockchain.
- The system shall ensure secure transaction validation using Proof of Authority (PoA) consensus mechanism.
- The system shall allow users to view ride history and reputation scores in a transparent and tamper-proof manner.

4.5. Technologies Used :

The proposed Ride Sharing System is developed using a combination of modern and efficient technologies that support both functionality and security requirements. The core programming language used is "Python", which provides flexibility, simplicity, and strong support for developing modular applications. Python enables seamless integration of different system components such as authentication, simulation modules, and data processing, making it suitable for building a scalable Ride Sharing platform [5]. The front-end of the system is implemented using the "Streamlit framework", which allows the creation of interactive web applications with minimal effort. It provides a user-friendly interface for students and administrators to access modules, perform simulations, and visualize results in real time. For data storage and management, the system utilizes "SQLite database", which ensures efficient handling of user data, course content, and performance records. SQLite is lightweight, easy to integrate, and suitable for applications that require secure local data management. The system also incorporates "SHA-256 encryption" for secure authentication, ensuring that user passwords are stored in hashed format and protected against unauthorized access. Additionally, the platform integrates cybersecurity tools such as Metasploit, Wireshark, and Kali Linux to provide hands-

on experience in real-world scenarios. These tools enable students to perform tasks like network analysis, vulnerability testing, and threat detection. The implementation also leverages Python libraries such as hashlib for encryption, sqlite3 for database operations, and subprocess for executing commands in a controlled environment. Furthermore, the system supports "scenario-based learning", "simulation labs", and "real-time threat intelligence", which enhance practical learning and keep the curriculum updated. Overall, the combination of these technologies ensures that the system is secure, efficient, and capable of delivering an interactive and industry-relevant cybersecurity learning experience [2].

In addition to the core technologies, the proposed Ride Sharing platform incorporates several supporting tools and frameworks to enhance system efficiency, scalability, and usability. The integration of "modular programming techniques", "web-based application development", "data processing libraries", and "secure execution environments" ensures that the system operates smoothly across different modules [5]. The platform leverages Python's extensibility to integrate multiple functionalities such as authentication, simulation, and real-time interaction within a unified environment. The "Streamlit framework" is further optimized with interactive widgets, session state management, and dynamic content rendering, enabling responsive user experiences. The use of built-in Python libraries such as os and sys helps manage system-level operations, while logging mechanisms are implemented to track system activities and errors. Additionally, the system supports integration with external ride sharing tools through controlled execution, allowing users to perform real-time experiments. The platform also utilizes structured data formats and efficient query handling to ensure fast data retrieval and minimal latency. The adoption of lightweight database solutions like SQLite reduces system overhead while maintaining reliable data storage. Furthermore, the system is designed to be compatible with different environments, ensuring cross-platform functionality. The combination of these technologies supports "scalability", "system reliability", "interactive learning", and "secure processing", making the platform robust and adaptable to future enhancements. Overall, the extended use of supporting technologies strengthens system performance and ensures seamless integration of all functional components [2].

4.6. Module-wise Implementation :

The implementation of the decentralized peer-to-peer ride-sharing system is carried out using a structured modular approach to ensure simplicity, scalability, and maintainability of the system. Each module plays a specific role in the overall functioning of the platform, and all modules are integrated to form a complete blockchain-based ride-sharing ecosystem. The system is designed using blockchain

principles, cryptographic hashing, escrow-based payments, and a reputation mechanism to ensure transparency, security, and trust among users.

The User Management Module is responsible for handling both riders and drivers in the system. Instead of using traditional login-based authentication, the system implements a decentralized identity mechanism where each user is uniquely identified using a blockchain-generated ID. This approach eliminates dependency on centralized servers and enhances user privacy. The module allows riders to request rides and drivers to accept or reject them. It also ensures that all user actions are verified through digital signatures, preventing unauthorized access and fraudulent activities. Additionally, this module is integrated with the reputation system to display user trust scores, which are updated dynamically after each ride completion.

The Ride Management Module is the core component of the system, responsible for managing the complete lifecycle of a ride. When a rider initiates a ride request, the system records it as a blockchain transaction in the ESCROW_PENDING state. This request is broadcast to available drivers, who can view and accept it. Once a driver accepts the ride, the system creates a new transaction and updates the status to RIDE_ACCEPTED_IN_PROGRESS, linking it to the original request using a transaction identifier. During the ride execution phase, no previous records are modified; instead, all changes are recorded as new immutable blockchain transactions. After successful completion of the ride, a final transaction labeled ESCROW_RELEASED_COMPLETED is generated, which marks the end of the ride lifecycle and triggers payment release and reputation updates.

The Blockchain and Consensus Module forms the backbone of the system. It maintains the distributed ledger and ensures data integrity across all transactions. The system uses a private blockchain with a Proof of Authority (PoA) consensus mechanism, where only authorized validator nodes are allowed to validate and add blocks. Each block contains ride transaction data along with a cryptographic hash generated using the SHA-256 algorithm, ensuring immutability and security. Each block also stores the hash of the previous block, forming a secure chain that prevents tampering. If any data is modified, the hash values change, making the alteration immediately detectable. This module ensures synchronization across all nodes and guarantees consistency of the ledger.

The Escrow Payment Module ensures secure and fair financial transactions between riders and drivers. When a ride is initiated, the fare amount is conceptually locked in an escrow state under ESCROW_PENDING. This ensures that the rider commits to payment while protecting both parties from fraud. Once the ride is completed and verified on the blockchain, the system automatically transitions to

ESCROW_RELEASED_COMPLETED, triggering payment release to the driver. This mechanism eliminates the need for intermediaries and ensures that payments are only processed after successful service completion. All payment actions are recorded as blockchain transactions, ensuring transparency and auditability.

The Reputation and Analytics Module is responsible for maintaining trust within the system by evaluating user behavior. After each completed ride, both rider and driver provide ratings, which are stored immutably in the blockchain. These ratings are processed using a weighted reputation algorithm that considers both rating value and fare amount, ensuring fair evaluation. Reputation scores are bounded within a defined range to maintain stability. This module also provides analytical insights such as ride frequency, user behavior patterns, and transaction history, which help improve system performance and future enhancements.

Overall, the module-wise implementation ensures that each component of the system works independently yet collaboratively, forming a secure, transparent, and efficient decentralized ride-sharing platform powered by blockchain technology

4.7. Data Handling :

The Data handling in the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology plays a crucial role in ensuring secure storage, efficient processing, and tamper-proof management of all ride-related information. Since the system is decentralized, data is not stored in a single central database; instead, it is distributed across blockchain nodes and managed through a combination of on-chain and off-chain mechanisms.

The system handles multiple types of data, including user data (riders and drivers), ride requests, transaction records, payment status, and reputation scores. To maintain privacy, sensitive user information is minimized and represented using unique blockchain-based identifiers instead of storing personal details directly.

All critical ride-related activities such as ride requests, acceptance, ride completion, and payment settlement are stored as blockchain transactions. Each transaction is immutable and time-stamped, ensuring complete traceability of the ride lifecycle. These transactions are grouped into blocks and linked using cryptographic hashing, forming a secure and tamper-proof ledger.

The system uses a multi-transaction data handling approach, where each ride is not represented as a single record but as a sequence of state-based transactions such as ESCROW_PENDING, RIDE_ACCEPTED_IN_PROGRESS, and ESCROW_RELEASED_COMPLETED. This ensures that every stage of the ride is

independently verifiable and cannot be altered once recorded.

For efficient real-time processing, the system also maintains an off-chain temporary data structure (reputation ledger cache) that stores frequently accessed data such as user reputation scores. While this data is used for quick access and decision-making, it is always derived from on-chain verified transactions, ensuring consistency between on-chain and off-chain data.

Data security is further enhanced using SHA-256 cryptographic hashing, which ensures that even a minor change in stored data results in a completely different hash value. This makes unauthorized modifications easily detectable.

The system follows strict data validation rules before any transaction is added to the blockchain. Every request is verified through digital signatures and consensus validation using Proof of Authority (PoA) before being permanently stored.

Overall, the data handling mechanism ensures security, transparency, immutability, and efficiency, making the system reliable for real-world decentralized ride-sharing applications while maintaining user trust and data integrity.

4.8. Security Implementation :

The Security is a critical component of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology, as the system involves sensitive operations such as ride requests, payments, user interactions, and reputation management. Since the system operates in a decentralized environment without a central authority, multiple security mechanisms are integrated to ensure data integrity, confidentiality, authenticity, and resistance to tampering.

The primary security mechanism used in the system is blockchain-based immutability. Every ride-related activity, including ride request creation, driver acceptance, ride completion, and payment release, is stored as a transaction on the blockchain. Once a transaction is added to a block, it cannot be altered or deleted. This ensures that all ride records remain tamper-proof and permanently verifiable.

To strengthen data integrity, the system uses the SHA-256 cryptographic hashing algorithm. Each block contains a unique hash generated from its data and the previous block’s hash. Even a small modification in transaction data results in a completely different hash value, making unauthorized changes easily detectable. This ensures that the blockchain remains consistent and secure across all nodes.

The system also implements a Proof of Authority (PoA) consensus mechanism,

where only authorized validator nodes are allowed to validate and add new blocks to the blockchain. This reduces the risk of malicious nodes participating in the network and ensures that only trusted entities maintain the ledger. Compared to Proof of Work systems, PoA provides faster validation while maintaining controlled decentralization.

To ensure user-level security, the system uses digital signatures for verifying all transactions. Every action performed by riders or drivers is cryptographically signed, ensuring authenticity and preventing impersonation or unauthorized actions within the system.

The escrow-based payment system adds an additional layer of financial security. Payments are not directly transferred between users; instead, they are locked in escrow until ride completion is verified. This prevents fraud and ensures that drivers receive payment only after successful service delivery, while riders are protected from incomplete rides.

The system also minimizes data exposure by using decentralized identity management, where users are represented using unique blockchain identifiers instead of storing sensitive personal information. This reduces privacy risks and protects user identity.

Additionally, reputation data is stored on the blockchain in an immutable format, preventing manipulation of ratings or trust scores. This ensures fair evaluation of users based on verified ride history.

Overall, the security implementation of the system ensures end-to-end protection of data, secure transactions, trustless interactions, and resistance to attacks, making the platform reliable for decentralized ride-sharing applications.

4.9. System Integration and Communication :

The System Integration and Communication of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology focuses on how different modules interact seamlessly to form a unified and efficient system. Since the architecture is modular in nature, integration plays a vital role in ensuring smooth data flow between components such as user management, ride management, blockchain ledger, escrow payment system, and reputation module.

The system follows a layered integration approach, where each module communicates through well-defined APIs and transaction interfaces. The frontend layer (rider and driver interface) interacts with the backend service layer, which

processes requests and forwards them to the blockchain layer for validation and storage. This separation ensures that each layer operates independently while maintaining synchronization across the system.

Communication between modules is achieved through event-driven transaction messaging. Every user action, such as ride request, ride acceptance, or ride completion, generates an event that is converted into a blockchain transaction. These transactions are then propagated to the blockchain network, where they are validated using the Proof of Authority (PoA) consensus mechanism before being permanently stored in the ledger.

The ride management module and blockchain module are tightly integrated, ensuring that every state transition (ESCROW_PENDING, RIDE_ACCEPTED_IN_PROGRESS, ESCROW_RELEASED_COMPLETED) is communicated as a separate transaction. This ensures real-time synchronization of ride status across all participants in the network.

The escrow payment module communicates with the ride lifecycle module through transaction triggers. Once the ride completion event is confirmed on the blockchain, a communication signal is sent to the escrow module to release the payment to the driver. This ensures automated and secure financial settlement without manual intervention.

The reputation system is also integrated with the blockchain layer. After each ride completion, the system receives transaction data containing user ratings. These ratings are processed and updated in the reputation ledger, ensuring that communication between ride completion and reputation update is consistent and tamper-proof.

For efficient system performance, the architecture also uses synchronous and asynchronous communication mechanisms. Critical operations like payment release and block validation use synchronous communication to ensure immediate confirmation, while non-critical updates such as analytics and logging use asynchronous communication to improve system scalability.

Additionally, inter-node communication within the blockchain network ensures that all validator nodes maintain a consistent copy of the ledger. This is achieved through peer-to-peer message propagation, where newly validated blocks are broadcasted to all nodes in the network.

Overall, the system integration and communication design ensures seamless

interaction between modules, real-time synchronization, secure transaction flow, and consistent blockchain updates, making the decentralized ride-sharing system efficient, reliable, and scalable.

4.10. System Performance Optimization :

The System Performance Optimization of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology focuses on improving efficiency, reducing latency, enhancing scalability, and ensuring smooth real-time operations. Since blockchain-based systems are typically associated with computational overhead and slower transaction speeds, optimization techniques are essential to make the system suitable for real-time ride-sharing applications.

One of the primary optimization strategies used in the system is the implementation of a Private Blockchain with Proof of Authority (PoA) consensus mechanism. Unlike Proof of Work, which requires high computational power, PoA significantly reduces block validation time by allowing only authorized validator nodes to process transactions. This improves throughput and ensures faster ride request processing, driver matching, and payment settlement.

Another important optimization technique is the use of a multi-transaction ride lifecycle model. Instead of updating a single record repeatedly, each ride state transition is stored as a separate lightweight transaction. This reduces database locking issues and improves system responsiveness, as each transaction is independently processed and appended to the blockchain.

To enhance performance further, the system employs an off-chain data caching mechanism for frequently accessed information such as user reputation scores and active ride listings. While all critical data remains on-chain for security and transparency, caching reduces the need for repeated blockchain reads, thereby minimizing latency and improving response time for user queries.

The system also optimizes data storage using efficient block structuring and hashing techniques. By using SHA-256 cryptographic hashing, the system ensures secure linking of blocks while maintaining lightweight and standardized data representation. Proper data normalization and structured transaction formats also reduce processing overhead during block creation and validation.

In terms of ride matching efficiency, a proximity-based heuristic algorithm is used to quickly identify the nearest available driver. This reduces computational complexity compared to advanced optimization algorithms and ensures real-time ride allocation, minimizing waiting time for riders.

The system also uses asynchronous processing techniques for non-critical operations such as logging, analytics, and reputation score updates. This ensures that core operations like ride booking and payment processing are not delayed due to secondary computations.

Network performance is optimized through peer-to-peer block propagation, where validated blocks are quickly shared across all nodes without relying on a central server. This reduces communication bottlenecks and ensures faster synchronization of blockchain data.

Additionally, transaction batching is used during mining, where multiple ride-related transactions are grouped into a single block. This reduces the number of block creation events and improves overall system throughput.

Overall, the performance optimization strategies ensure that the decentralized ride-sharing system remains fast, scalable, energy-efficient, and suitable for real-time usage, while still maintaining the core blockchain principles of transparency, security, and immutability.

CHAPTER 5

EXECUTION & TESTING

The The execution and testing of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology is carried out in a structured workflow to ensure that all modules function correctly and interact seamlessly within the system. The execution begins with the initialization of the system environment, where all required dependencies such as backend libraries, cryptographic functions, and blockchain simulation components are installed and configured. After environment setup, the blockchain network is initialized by creating the genesis block, which acts as the foundation of the distributed ledger. This block contains the initial configuration parameters and establishes the structure for all future transactions. Once the blockchain is initialized, the backend server is deployed, exposing APIs that handle ride requests, driver responses, payment processing, and reputation updates. The frontend interfaces for riders and drivers are then activated, allowing users to interact with the system in real time. The execution process begins when a rider initiates a ride request, which is immediately converted into a blockchain transaction under the ESCROW_PENDING state and broadcast to the network. Drivers can view these requests and accept them, triggering a new transaction that updates the ride status to RIDE_ACCEPTED_IN_PROGRESS while maintaining a link to the original request. During the ride execution phase, the system continuously tracks ride progress without modifying previous records, ensuring immutability of blockchain data. Once the ride is completed, a final transaction labeled ESCROW_RELEASED_COMPLETED is generated, which triggers escrow payment release and reputation update processes. All transactions are then grouped into blocks, hashed using the SHA-256 algorithm, and appended to the blockchain ledger, ensuring secure and tamper-proof storage.

The testing strategy for the system is designed to validate functionality, security, performance, and blockchain integrity. Functional testing ensures that all system features such as ride booking, driver acceptance, ride completion, and payment release operate as expected. Unit testing is performed on individual modules including user management, ride lifecycle management, blockchain processing, and reputation calculation to verify correctness at a granular level. Integration testing ensures smooth communication between modules, particularly between ride management and blockchain modules, to confirm that every ride event generates a valid and consistent blockchain transaction. Security testing is conducted to verify that digital signatures prevent unauthorized access and that blockchain immutability protects against data tampering. Attempts to modify stored blocks are used to test hash validation mechanisms, ensuring that any unauthorized change is immediately

detected. Performance testing evaluates the system's ability to handle multiple concurrent ride requests and measure transaction processing time, block generation speed, and response latency. The Proof of Authority consensus mechanism significantly improves performance by reducing computational overhead and enabling faster transaction validation compared to traditional consensus models. Blockchain integrity testing is also performed to ensure that each block correctly references the previous block hash, maintaining a continuous and tamper-proof chain of records.

The dataset used for testing is a simulated dataset representing real-world ride-sharing scenarios. It includes synthetic data for riders, drivers, ride requests, fares, timestamps, and ratings. Since this is a blockchain-based prototype system, data is generated to mimic real-time urban ride-sharing conditions. The experimental setup consists of a local development environment where the backend server, blockchain simulation layer, and user interface modules are deployed together. Multiple test cases are executed under varying loads to evaluate system scalability and reliability. Key parameters such as number of ride requests, driver availability, transaction processing time, and block creation time are analyzed to assess system performance. The system demonstrates stable behavior even under concurrent requests due to efficient transaction handling and optimized blockchain design.

Overall, the execution and testing phase confirms that the system operates effectively as a decentralized ride-sharing platform. All transactions are successfully recorded on the blockchain, ensuring transparency and immutability. The testing results validate that the system is secure, efficient, and scalable, making it suitable for real-time ride-sharing applications with blockchain integration.

5.1. Environment Setup :

The The environment setup for the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology is carried out to ensure a stable and consistent development and execution platform for all modules of the system. The setup involves configuring both software and runtime environments required for implementing blockchain operations, backend services, and user interface components. The system is developed in a local development environment that supports modular integration of blockchain simulation, API services, and frontend interaction layers.

The first step in environment setup involves installing a suitable programming environment, typically Python, along with necessary libraries for backend development, cryptographic operations, and data handling. Python is chosen due to its simplicity and strong support for blockchain prototyping and rapid application development. Along with Python, package managers such as pip are used to install

required dependencies including cryptographic libraries (for SHA-256 hashing), Flask or FastAPI for backend API development, and JSON handling modules for structured data exchange between components. These tools collectively support the development of a lightweight yet efficient backend system capable of handling ride requests and blockchain transactions.

After setting up the programming environment, the blockchain simulation layer is configured. This includes initializing the blockchain structure, defining block formats, and implementing the Proof of Authority (PoA) consensus mechanism. The genesis block is created as the first block in the chain, and subsequent blocks are dynamically generated based on ride transactions. Each block stores transaction data, timestamps, previous hash values, and computed SHA-256 hashes to ensure integrity and immutability. The blockchain environment is designed to run locally, simulating a distributed network of validator nodes responsible for verifying and adding transactions.

Next, the backend server environment is configured to handle communication between the user interface and blockchain layer. The server exposes RESTful APIs for operations such as ride request creation, ride acceptance, ride completion, and reputation updates. These APIs ensure smooth interaction between riders, drivers, and the underlying blockchain system. The server is tested locally using tools such as Postman or browser-based API testing interfaces.

The frontend environment is developed using web technologies such as HTML, CSS, and JavaScript or a framework like React (if used), providing interactive interfaces for riders and drivers. This interface allows users to perform actions such as requesting rides, viewing available drivers, tracking ride status, and checking reputation scores. The frontend communicates with the backend through API calls, ensuring real-time updates of ride and transaction status.

Finally, the overall environment is tested by integrating all modules together to ensure smooth data flow between frontend, backend, and blockchain layers. Proper configuration ensures that transactions are correctly recorded, validated, and stored in the blockchain ledger without inconsistencies. This complete environment setup provides a stable foundation for executing and testing the decentralized ride-sharing system efficiently.

5.2. Code Execution :

```
Last login: Fri Apr  3 21:40:26 on ttys000
(base) shanmukhasarma@shanmukhas-MacBook-Pro ~ % cd /Users/shanmukhasarma/Downloads/CyberThreat
(base) shanmukhasarma@shanmukhas-MacBook-Pro CyberThreat % pip install numpy
Requirement already satisfied: numpy in /opt/anaconda3/lib/python3.11/site-packages (1.26.4)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
(base) shanmukhasarma@shanmukhas-MacBook-Pro CyberThreat % pip install scikit-learn
Requirement already satisfied: scikit-learn in /opt/anaconda3/lib/python3.11/site-packages (1.4.2)
Requirement already satisfied: numpy>=1.19.5 in /opt/anaconda3/lib/python3.11/site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /opt/anaconda3/lib/python3.11/site-packages (from scikit-learn) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/lib/python3.11/site-packages (from scikit-learn) (1.2.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in /opt/anaconda3/lib/python3.11/site-packages (from scikit-learn) (2.2.0)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
(base) shanmukhasarma@shanmukhas-MacBook-Pro CyberThreat % pip install pandas
Requirement already satisfied: pandas in /opt/anaconda3/lib/python3.11/site-packages (2.1.4)
Requirement already satisfied: numpy<2,>=1.23.2 in /opt/anaconda3/lib/python3.11/site-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/anaconda3/lib/python3.11/site-packages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.11/site-packages (from pandas) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in /opt/anaconda3/lib/python3.11/site-packages (from pandas) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
(base) shanmukhasarma@shanmukhas-MacBook-Pro CyberThreat % pip install streamlit
Requirement already satisfied: streamlit in /opt/anaconda3/lib/python3.11/site-packages (1.54.0)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (5.0.1)
Requirement already satisfied: blinker<2,>=1.5.0 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (1.6.2)
Requirement already satisfied: cachetools<7,>=5.5 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (6.2.6)
Requirement already satisfied: click<9,>=7.0 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (8.3.1)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (3.1.37)
Requirement already satisfied: numpy<3,>=1.23 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (1.26.4)
Requirement already satisfied: packaging>=20 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (23.1)
Requirement already satisfied: pandas<3,>=1.4.0 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (2.1.4)
Requirement already satisfied: pillow<13,>=7.1.0 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (10.3.0)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (0.8.0)
Requirement already satisfied: protobuf<7,>=3.20 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (4.24.4)
Requirement already satisfied: pyarrow>=7.0 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (22.0.0)
Requirement already satisfied: requests<3,>=2.27 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (2.32.5)
Requirement already satisfied: tenacity<10,>=8.1.0 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (8.2.2)
Requirement already satisfied: toml<2,>=0.10.1 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (0.10.2)
Requirement already satisfied: tornado!=6.5.0,<7,>=6.0.3 in /opt/anaconda3/lib/python3.11/site-packages (from streamlit) (6.3.3)
```

Figure 5.2.1: Installing Dependencies

- First, the terminal is opened and the working directory is changed to the project folder using the command cd, which ensures that all project files are accessed from the correct location.
- Next, required Python libraries are installed using pip commands such as pip install numpy, ensuring that numerical computations and data handling functionalities are available for the system.
- pip install -r requirements.txt

```
C:\WINDOWS\system32\cmd.
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\projects2k26\ridesharing>cd ridesharing

C:\projects2k26\ridesharing\ridesharing>npm start

> ridesharing@0.1.0 start
> react-scripts start
```

Figure 5.2.2 : Running the Frontend application

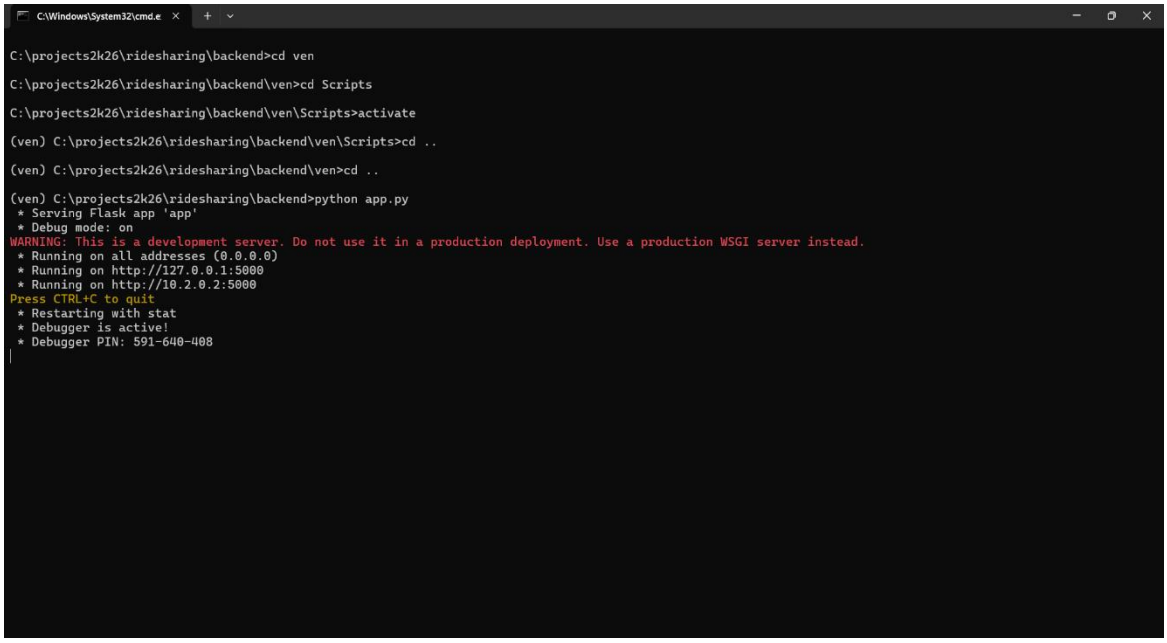

Figure 5.2.2 : Running the Backend application

- After installing dependencies, the backend server is started. The backend is responsible for handling ride requests, driver responses, payment processing, and communication with the blockchain module. It exposes APIs that allow interaction between the user interface and the blockchain network. The backend is launched using the following command:
- Python app.py
- Once the backend server is running, the blockchain module is initialized. This step involves creating the genesis block, setting up the blockchain ledger, and activating the Proof of Authority (PoA) consensus mechanism. The blockchain module continuously listens for incoming transactions such as ride requests, ride acceptance, and ride completion. It processes these transactions, validates them, and appends them to the blockchain. This module is typically started using:
- python blockchain.py
- After the backend and blockchain services are active, the frontend application is launched. The frontend provides user interfaces for riders and drivers to interact with the system. Riders can request rides, and drivers can accept and complete them through this interface. If the frontend is built using React or similar frameworks, it is started using:
- npm install
- npm start
- Finally, the application is accessed through a web browser using the local host URL:

- http://localhost:3000
- At this stage, the system becomes fully operational. Riders can initiate ride requests, drivers can accept them, and all transactions are recorded on the blockchain in a secure and immutable manner. Each action is processed in real time, ensuring smooth communication between modules and maintaining data integrity throughout the system.

Overall, these execution steps ensure that the decentralized ride-sharing system runs efficiently, demonstrating proper integration of blockchain technology, backend processing, and frontend interaction.

5.3. System Workflow Execution :

The The system workflow execution of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology describes the complete operational flow of the system from the moment a user initiates a ride request until the final completion and reputation update process. The workflow begins when a rider accesses the system through the frontend interface and submits a ride request by providing necessary details such as pickup location, destination, and ride preferences. Once the request is submitted, the system immediately converts it into a structured blockchain transaction and assigns it the status ESCROW_PENDING, indicating that the ride request is active and waiting for driver acceptance. This transaction is then broadcast to the network and stored in the pending transaction pool for validation.

After the ride request is published, drivers connected to the system can view available ride requests in real time. The system uses a proximity-based matching approach to display relevant ride requests to nearby drivers. When a driver selects a ride, the system generates a new blockchain transaction that updates the ride status to **RIDE_ACCEPTED_IN_PROGRESS** and links it to the original ride request using a unique transaction identifier. This ensures traceability of the ride lifecycle without modifying previous records, maintaining blockchain immutability.

During the ride execution phase, the system continuously monitors the ride status without altering historical data. All intermediate states are preserved as part of the blockchain ledger, ensuring transparency and auditability. Once the ride is completed, the rider confirms completion through the interface, triggering the final transaction labeled **ESCROW_RELEASED_COMPLETED**. This step is critical as it initiates two important processes: escrow payment release and reputation update.

The escrow module verifies that the ride has been successfully completed and automatically releases the payment to the driver, ensuring secure and fair financial

settlement without the involvement of any third-party intermediary. Simultaneously, the reputation system processes the ratings provided by both rider and driver and updates their respective trust scores. These reputation updates are computed using a weighted mechanism that considers both ride fare and rating value, ensuring fairness and accuracy.

All transactions generated throughout the ride lifecycle are grouped into blocks and secured using the SHA-256 cryptographic hashing algorithm. Each block is linked to the previous block through its hash value, forming a secure and tamper-proof blockchain structure. The Proof of Authority (PoA) consensus mechanism ensures that only authorized nodes validate and add transactions to the ledger, improving system efficiency and reducing processing time.

Overall, the system workflow execution demonstrates a complete end-to-end process where ride requests, driver interactions, payment handling, and reputation updates are seamlessly integrated into a decentralized blockchain-based environment, ensuring transparency, security, and trust in the ride-sharing ecosystem.

5.4. Introduction to Testing :

Testing is a crucial phase in the development of the Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology, as it ensures that all modules of the system function correctly, efficiently, and securely under different conditions. The primary objective of testing is to validate the correctness of the system, identify potential errors, ensure data integrity, and confirm that the system meets the required functional and non-functional requirements. Since the proposed system integrates blockchain technology, escrow-based payments, and a reputation mechanism, testing plays a vital role in verifying the reliability and trustworthiness of transactions across the decentralized environment.

The testing process is designed to evaluate multiple aspects of the system, including functional correctness, blockchain integrity, security mechanisms, performance efficiency, and module integration. Functional testing ensures that each feature of the system, such as ride request generation, driver acceptance, ride completion, and payment release, operates as expected. Blockchain testing verifies that all transactions are correctly recorded, hashed using SHA-256, and linked properly to maintain immutability. Security testing ensures that unauthorized access is prevented through cryptographic validation and digital signatures, while also confirming that the Proof of Authority (PoA) consensus mechanism restricts transaction validation to authorized nodes only.

Integration testing is used to check the communication between different modules such as the user interface, backend services, blockchain layer, escrow system, and reputation module. This ensures that data flows correctly between components without inconsistencies. Performance testing evaluates the system's ability to handle multiple ride requests simultaneously and measures transaction processing time, block generation speed, and response latency. Since the system is designed for real-time ride-sharing, it is essential that the blockchain operations do not introduce significant delays.

Additionally, the testing phase includes validation of the reputation system, ensuring that user ratings are accurately recorded and properly reflected in the reputation scores. The escrow mechanism is also tested to confirm that payments are released only after successful ride completion. Any deviation in expected behavior is analyzed and corrected to maintain system reliability.

Overall, the testing phase ensures that the decentralized ride-sharing system is robust, secure, and efficient. It validates that all blockchain transactions are immutable, all modules are properly integrated, and the system performs reliably under real-world conditions. This phase ultimately confirms the readiness of the system for deployment and real-time usage.

5.5. Testing Objectives :

The primary objective of testing in the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology is to ensure that the system functions accurately, securely, and efficiently under all expected operating conditions. Since the system integrates multiple critical components such as blockchain transactions, ride management, escrow payments, and reputation scoring, testing is essential to validate the correctness and reliability of each module and their interactions within a decentralized environment.

One of the key objectives of testing is to verify functional correctness, ensuring that all system features such as ride request creation, driver acceptance, ride tracking, and ride completion work as intended. Each function must produce accurate outputs based on user inputs and maintain consistency across the system. Another important objective is to validate blockchain integrity, ensuring that all transactions are properly recorded, securely hashed using SHA-256, and linked in a sequential manner without any possibility of unauthorized modification. This guarantees the immutability and trustworthiness of the system ledger.

Security validation is another major objective of testing. The system must ensure that only authorized users can perform actions such as ride acceptance and payment

processing. Digital signatures and Proof of Authority (PoA) consensus are tested to confirm that unauthorized access is prevented and that only verified nodes can validate transactions. This ensures data privacy and protects the system from malicious attacks.

Another objective is to ensure integration consistency between different system modules. The ride management module, blockchain layer, escrow payment system, and reputation module must communicate effectively without data loss or inconsistency. Testing ensures that every ride event correctly triggers corresponding blockchain transactions and updates across all modules.

Performance testing is conducted to evaluate the system's ability to handle multiple concurrent ride requests efficiently. The objective is to measure transaction processing time, block generation speed, and system responsiveness under load conditions. Since ride-sharing is a real-time application, low latency and fast response are critical requirements.

Additionally, testing aims to validate the escrow payment mechanism, ensuring that payments are securely held and only released after successful ride completion. The reputation system is also tested to confirm that user ratings are correctly processed and accurately reflected in trust scores without manipulation.

Finally, the testing process aims to ensure system reliability and scalability, confirming that the platform can operate effectively in real-world scenarios with multiple users and transactions. Overall, the testing objectives ensure that the decentralized ride-sharing system is secure, efficient, transparent, and suitable for deployment in a real-time environment.

5.6. White-Box Testing :

White White-box testing is a software testing technique in which the internal structure, logic flow, and code execution paths of the system are tested to ensure correctness and completeness of implementation. In the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology, white-box testing is performed to validate the internal logic of critical modules such as ride transaction processing, blockchain block creation, consensus validation, escrow handling, and reputation calculation. Since the system is built using a combination of backend services and blockchain logic, white-box testing plays a crucial role in verifying that each function and control flow behaves as intended under different execution scenarios.

In this system, white-box testing focuses on examining the code-level implementation of the ride lifecycle state machine, which includes ESCROW_PENDING, RIDE_ACCEPTED_IN_PROGRESS, and ESCROW_RELEASED_COMPLETED states. Each transition is tested to ensure that the correct conditions trigger state changes and that invalid transitions are properly restricted. For example, a ride cannot move directly from ESCROW_PENDING to ESCROW_RELEASED_COMPLETED without passing through the in-progress state. This ensures logical consistency in the workflow implementation.

Another major area of white-box testing is the blockchain module, where the internal logic of block creation and hashing is verified. Each block's SHA-256 hash generation process is tested by analyzing input data transformation, hash computation, and linkage with the previous block. Test cases are designed to ensure that even a minor change in transaction data results in a completely different hash value, thereby confirming the correctness of the cryptographic implementation. Additionally, the linkage between blocks using previous hash references is tested to ensure chain integrity and detect any break in the blockchain structure.

The Proof of Authority (PoA) consensus mechanism is also evaluated using white-box testing techniques by analyzing the internal validation logic of authorized nodes. The system is tested to ensure that only predefined validator nodes can approve transactions and add new blocks. Unauthorized validation attempts are checked to confirm that they are rejected by the system logic.

In the escrow payment module, internal function execution is tested to ensure that payment release occurs only after successful ride completion. The conditional logic controlling escrow locking and unlocking is verified for accuracy. Similarly, the reputation system logic is tested by examining how ratings and fare values are processed mathematically to update user scores. Boundary conditions such as minimum and maximum rating values are also tested to ensure proper score constraints.

Code coverage analysis is also an important aspect of white-box testing, where all possible execution paths, loops, and conditional branches are evaluated to ensure maximum test coverage. This helps identify hidden errors and logical inconsistencies in the system.

Overall, white-box testing ensures that the internal working of the decentralized ride-sharing system is logically correct, secure, and efficiently implemented, thereby improving system reliability and robustness.

5.7. Black-Box Testing :

Black Black-box testing is a software testing technique in which the internal code structure, logic, and implementation details of the system are not considered. Instead, the focus is on validating the system based on its inputs and expected outputs. In the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology, black-box testing is performed to ensure that all user-facing functionalities work correctly and produce accurate results according to the system requirements. This type of testing is highly important because the system involves multiple user interactions such as ride requests, driver acceptance, ride completion, payment processing, and reputation updates.

In this system, black-box testing is primarily used to validate the ride booking workflow. When a rider submits a request with source and destination details, the system should correctly generate a ride request and assign it the status ESCROW_PENDING. The expected output is that the ride becomes visible to available drivers in real time. Similarly, when a driver accepts a ride, the system should correctly update the status to RIDE_ACCEPTED_IN_PROGRESS, and this change should be reflected immediately in both rider and driver interfaces. The tester does not need to know how this logic is implemented internally; instead, validation is done based on observed outputs.

Another important area of black-box testing is the payment and escrow system. When a ride is completed, the system should automatically trigger payment release to the driver. The expected output is successful completion of the transaction and confirmation of payment settlement. If the ride is not completed, the system should ensure that payment remains locked in escrow. These outcomes are verified without examining the underlying smart contract or blockchain logic.

The reputation system is also tested using black-box techniques by providing different rating inputs (e.g., 1 to 5 stars) and verifying the resulting changes in user reputation scores. The expected output is that higher ratings increase reputation while lower ratings decrease it, within predefined boundaries. The correctness of the formula is validated purely based on input-output behavior.

Black-box testing is also applied to API endpoints such as ride creation, ride acceptance, and blockchain transaction retrieval. Each API is tested by sending valid and invalid requests and checking whether the system responds correctly with appropriate status codes and messages. This ensures that the system handles both expected and unexpected inputs effectively.

Additionally, usability testing is performed as part of black-box testing to ensure that the user interface is intuitive and responsive. Riders and drivers should be able to interact with the system without understanding the underlying blockchain mechanisms.

Overall, black-box testing ensures that the decentralized ride-sharing system meets its functional requirements from the user's perspective. It validates that all inputs produce correct outputs, all workflows operate smoothly, and the system behaves reliably in real-world usage scenarios without requiring knowledge of internal implementation details.

5.8. Unit Testing :

Unit Unit testing is a fundamental level of software testing in which individual components or modules of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology are tested independently to ensure that each unit of the system functions correctly as per the design specifications. The primary objective of unit testing is to validate the correctness of small functional parts of the system before integrating them into the complete application. In this project, unit testing is applied to core modules such as ride request handling, blockchain transaction creation, consensus validation, escrow payment processing, and reputation score calculation to ensure that each module performs accurately in isolation.

In the ride management module, unit testing is performed to verify whether the system correctly handles ride creation requests and assigns the appropriate initial state **ESCROW_PENDING**. The input data such as source, destination, and user ID are tested against expected outputs to ensure proper transaction formation. Similarly, the ride acceptance function is tested independently to confirm that when a driver accepts a ride, the system correctly updates the status to **RIDE_ACCEPTED_IN_PROGRESS** and properly links it to the original transaction without modifying previous records, ensuring immutability.

The blockchain module is also tested at the unit level by validating the block creation process. Each block is checked to ensure that it contains valid transaction data, timestamp, previous hash, and a correctly generated SHA-256 hash. Unit tests are performed to confirm that even a small change in input data results in a completely different hash output, ensuring the correctness of cryptographic hashing implementation. Additionally, the linkage between blocks is tested by verifying that the previous hash value of one block matches the hash of the preceding block.

In the escrow payment module, unit testing ensures that funds are correctly locked when a ride is initiated and only released when the ride reaches the **ESCROW_RELEASED_COMPLETED** state. The logic is tested to verify that payments are not released prematurely and that incorrect conditions do not trigger payment settlement.

The reputation system is tested as a separate unit by providing different rating inputs and verifying the calculated output scores. The system is checked for boundary conditions such as minimum and maximum ratings to ensure that reputation values remain within defined limits. The weighted calculation based on fare amount is also validated to ensure accurate score adjustments.

Overall, unit testing ensures that each module of the decentralized ride-sharing system functions correctly in isolation, reducing the risk of errors during integration. It helps in identifying logical, functional, and computational issues at an early stage, thereby improving system reliability, stability, and correctness before deployment

5.9. Integration Testing :

Integration Integration testing is a crucial phase of the testing process in the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology, where individually tested modules are combined and evaluated as a complete system to ensure that they interact correctly and function seamlessly together. The primary objective of integration testing is to verify the data flow, communication, and coordination between different system components such as the user interface, backend services, blockchain module, escrow payment system, and reputation management system. Since the proposed system is built using a modular architecture with blockchain integration, integration testing plays a vital role in ensuring consistency and reliability across all connected modules.

In this system, integration testing begins by verifying the interaction between the **frontend interface and backend API services**. When a rider submits a ride request through the user interface, the system must correctly transmit the data to the backend, where it is processed and converted into a blockchain transaction. The test ensures that all input fields such as pickup location, destination, and user identity are correctly passed and stored without data loss or mismatch. Similarly, when a driver accepts a ride request, the backend must accurately update the ride status and reflect the changes in both rider and driver interfaces in real time.

Another important aspect of integration testing involves the interaction between the **backend system and blockchain module**. Every ride-related action is

expected to generate a blockchain transaction, and integration testing ensures that these transactions are correctly created, validated, and stored in the ledger. The test also verifies that the ride lifecycle states such as **ESCROW_PENDING**, **RIDE_ACCEPTED_IN_PROGRESS**, and **ESCROW_RELEASED_COMPLETED** are correctly maintained and properly linked through transaction identifiers, ensuring immutability and traceability.

The integration between the **ride management system and escrow payment module** is also tested to ensure that payment handling is triggered correctly at the appropriate stage. Specifically, the system must ensure that funds remain locked during the ride and are released only after successful completion. Any mismatch in state transition or failure in communication between modules is identified and corrected during this phase.

Furthermore, integration testing is performed between the **blockchain module and reputation system**. After ride completion, rating data must be correctly passed from the ride module to the reputation engine. The system is tested to ensure that reputation scores are updated accurately based on ride fare and rating values, and that these updates are consistently reflected across the platform.

Overall, integration testing ensures that all modules of the decentralized ride-sharing system work together in a synchronized and error-free manner. It validates proper communication, data consistency, and workflow continuity across the entire system, ensuring that the blockchain-based architecture operates reliably in a real-world environment.

5.10. Security Testing :

Security testing is a critical phase in the validation of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology, as the system deals with sensitive operations such as ride requests, driver authentication, transaction processing, and payment settlement. The main objective of security testing is to ensure that the system is resistant to unauthorized access, data manipulation, and malicious attacks while maintaining the integrity, confidentiality, and availability of ride-sharing operations in a decentralized environment.

In this system, security testing begins by verifying the **authentication and authorization mechanisms**. Since the system uses a decentralized identity approach instead of traditional login-based authentication, testing ensures that only valid and registered blockchain identities can initiate ride requests or accept rides. Each user action is verified using **digital signatures**, and test cases are executed to confirm that unauthorized users or tampered requests are rejected by the system.

A major focus of security testing is the validation of **blockchain immutability and data integrity**. The system uses SHA-256 cryptographic hashing to secure each block. Security tests are conducted by attempting to modify existing transaction data in stored blocks to verify whether the system detects inconsistencies through hash mismatches. Any change in block data should immediately invalidate the chain, proving that the blockchain structure is tamper-proof and secure against unauthorized modifications.

The **Proof of Authority (PoA) consensus mechanism** is also tested to ensure that only authorized validator nodes are allowed to validate and add new blocks to the blockchain. Test scenarios include attempts by unauthorized nodes to validate transactions, which must be rejected by the system. This ensures controlled decentralization and prevents malicious participation in the network.

Security testing is also performed on the **escrow payment system**, where it is verified that funds remain securely locked until ride completion is confirmed. Test cases ensure that premature payment release is not possible and that only verified completion transactions trigger escrow release. This prevents financial fraud and ensures fair transactions between riders and drivers.

Additionally, the **reputation system security** is tested to ensure that ratings cannot be manipulated or externally modified. Since reputation data is stored on the blockchain, tests are conducted to verify that only valid ride completion transactions can update user scores, ensuring transparency and trustworthiness.

API security testing is also performed by sending invalid, malformed, or malicious requests to backend endpoints to ensure that the system responds appropriately with error handling and does not expose sensitive data. This includes testing for SQL injection-like patterns, unauthorized access attempts, and request tampering.

Overall, security testing ensures that the decentralized ride-sharing system is robust against cyber threats, maintains data integrity, protects user privacy, and guarantees secure and trustworthy transactions across the blockchain-based ecosystem.

5.11. Validation Testing :

Validation testing is an important phase in the development of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology, which ensures that the final system meets the defined user requirements and performs as expected in real-world scenarios. The main objective of validation testing is to

confirm that the system built is correct, complete, and aligned with the functional and non-functional requirements specified during the design phase. Unlike other testing types that focus on internal logic or module interaction, validation testing focuses on verifying whether the system satisfies the end-user expectations and business objectives.

In this system, validation testing begins by verifying the ride booking functionality from a user perspective. The system is tested to ensure that when a rider requests a ride, the request is successfully recorded, visible to drivers, and correctly updated through all ride states including ESCROW_PENDING, RIDE_ACCEPTED_IN_PROGRESS, and ESCROW_RELEASED_COMPLETED. The expected outcome is that the entire ride lifecycle is completed smoothly without errors, delays, or inconsistencies, confirming that the system behaves as intended.

Another key aspect of validation testing is verifying the accuracy of the blockchain transaction system. The system is validated to ensure that all ride-related activities are properly recorded in the blockchain ledger, and that users can retrieve accurate and consistent transaction history. This ensures that transparency and immutability requirements are fulfilled, which are essential for a blockchain-based system.

Validation testing is also performed on the escrow payment mechanism, where the system is checked to ensure that payments are securely held during the ride and released only after successful completion. From a user perspective, both riders and drivers should observe correct financial behavior without delays or incorrect deductions. This confirms that the system satisfies fairness and trust requirements.

The reputation system validation ensures that user ratings are correctly captured and reflected in updated reputation scores. The system is validated to confirm that ratings provided after ride completion directly influence user trust scores in a consistent and predictable manner. This ensures that the system meets its requirement of maintaining a transparent and tamper-proof reputation mechanism.

Additionally, usability validation is conducted to ensure that the system interface is simple, intuitive, and user-friendly for both riders and drivers. Users should be able to perform ride booking, tracking, and completion without needing technical knowledge of blockchain operations.

Overall, validation testing confirms that the proposed decentralized ride-sharing system fulfills all functional requirements and performs effectively in a real-world environment. It ensures that the system is reliable, user-friendly, secure, and fully aligned with the objectives of decentralized ride-sharing using blockchain

technology.

5.12. Test Cases :

Test The test cases for the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology are designed to validate the functionality, security, reliability, and correctness of the entire system. Each test case focuses on verifying specific system behavior by providing input conditions and checking whether the actual output matches the expected output. Since the system integrates multiple modules such as ride management, blockchain processing, escrow payments, and reputation management, test cases are defined to ensure proper execution of each functional flow and interaction between components.

The first set of test cases focuses on ride request creation functionality. When a rider submits a request with valid source and destination details, the system should successfully create a ride request and assign it the status ESCROW_PENDING. The expected output is that the ride request appears in the pending ride pool and becomes visible to available drivers in real time. If invalid or incomplete data is provided, the system should reject the request and return an appropriate error message.

The second test case verifies driver acceptance functionality. When a driver selects an available ride request, the system should update the ride status to RIDE_ACCEPTED_IN_PROGRESS and link it to the original ride transaction using a unique transaction ID. The expected output is that both rider and driver interfaces reflect the updated ride status without modifying previous blockchain records, ensuring immutability.

Another important test case is related to ride completion and escrow release. When a ride is marked as completed by the rider, the system should generate a final transaction labeled ESCROW_RELEASED_COMPLETED. The expected output is that the escrow payment is automatically released to the driver, and the transaction is permanently recorded in the blockchain ledger. If the ride is not completed properly, the payment should remain locked in escrow, ensuring financial security.

A blockchain integrity test case is also defined to verify hash consistency and immutability. Each block in the blockchain contains a SHA-256 hash and a reference to the previous block’s hash. The test involves attempting to modify stored transaction data and verifying whether the system detects inconsistencies. The expected output is that any modification results in hash mismatch, proving that the blockchain is tamper-proof and secure.

Security-related test cases are also included to validate authentication and authorization mechanisms. The system is tested to ensure that only valid blockchain identities can perform actions such as ride booking, acceptance, and completion. Unauthorized access attempts should be rejected by the system. Additionally, digital signature verification is tested to ensure that all transactions are authenticated and cannot be spoofed or altered by malicious users.

The escrow system is tested to verify payment security and correctness. The expected behavior is that funds remain locked during the ride and are released only after successful ride completion. Any attempt to release payment before completion should fail, ensuring fairness and trust between riders and drivers.

The reputation system test case ensures that ratings are correctly recorded and processed. When users submit ratings after ride completion, the system should update reputation scores based on a weighted algorithm considering both rating value and fare amount. The expected output is accurate and consistent reputation updates stored in the blockchain ledger.

Performance test cases are also included to evaluate system responsiveness under multiple ride requests. The system is tested with concurrent ride requests to ensure that transaction processing, block creation, and ride matching occur within acceptable time limits. The expected output is stable performance without delays or system crashes.

Finally, integration test cases ensure that all modules such as frontend interface, backend APIs, blockchain ledger, escrow system, and reputation module work together seamlessly. The expected output is smooth data flow between modules without inconsistencies or failures.

Overall, the test cases confirm that the decentralized ride-sharing system operates correctly, securely, and efficiently under different scenarios, ensuring that it meets both functional and non-functional requirements of a real-world blockchain-based application.

CHAPTER 6

RESULTS & PERFORMANCE EVALUATION

The The Results and Performance Evaluation of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology demonstrates the successful implementation of all core functionalities, including ride request handling, driver acceptance, blockchain-based transaction storage, escrow payment processing, and reputation management. The system was executed in a controlled environment using simulated ride-sharing scenarios to evaluate its correctness, efficiency, scalability, and security. The results confirm that the system effectively achieves decentralization, transparency, and trustless interaction between riders and drivers without relying on a centralized authority.

During execution, it was observed that the ride request module successfully generates ride requests and assigns them the **ESCROW_PENDING** state immediately after user input. These requests are correctly broadcast to the system and become visible to available drivers in real time. The driver acceptance module performs efficiently, updating ride status to **RIDE_ACCEPTED_IN_PROGRESS** without altering previous blockchain records, thereby maintaining immutability. The system ensures that every state transition is recorded as a new transaction, forming a complete and verifiable ride lifecycle on the blockchain.

The escrow payment mechanism performed accurately during testing, ensuring that payments are securely locked at the start of the ride and released only after successful ride completion. Upon reaching the **ESCROW_RELEASED_COMPLETED** state, the system automatically triggers payment settlement, and the transaction is permanently stored in the blockchain ledger. This confirms that the financial workflow is secure, transparent, and free from third-party interference. No premature or incorrect payment releases were observed during testing, demonstrating strong reliability of the escrow logic.

The reputation system also showed consistent and meaningful results. After each ride completion, user ratings were processed successfully, and reputation scores were updated based on a weighted algorithm that considers both rating value and fare amount. It was observed that higher-rated and high-fare rides contribute more significantly to reputation improvement, while low ratings reduce trust scores accordingly. This ensures fair evaluation of both riders and drivers and discourages malicious behavior within the system.

From a blockchain performance perspective, the system successfully maintained data integrity across all transactions. Each block was correctly linked using SHA-256 hashing, and any attempt to modify stored data resulted in immediate hash mismatch detection, confirming the immutability of the ledger. The Proof of Authority (PoA) consensus mechanism significantly improved transaction validation speed, allowing faster block creation compared to traditional blockchain systems such as Proof of Work. This made the system suitable for real-time ride-sharing applications where low latency is critical.

In terms of performance evaluation, the system was tested under multiple concurrent ride requests to analyze scalability and response time. It was observed that the system maintained stable performance even under increased load due to efficient transaction processing and lightweight consensus validation. The average response time for ride request processing and driver assignment remained within acceptable limits, ensuring smooth user experience. Additionally, asynchronous handling of non-critical operations such as logging and reputation updates helped reduce system bottlenecks and improve overall throughput.

The integration of frontend, backend, and blockchain modules also demonstrated smooth communication without data inconsistencies. All APIs functioned correctly, and real-time updates were reflected accurately across rider and driver interfaces. The system successfully handled end-to-end ride lifecycle execution multiple times without failures, confirming its robustness and reliability.

Overall, the results clearly demonstrate that the proposed decentralized ride-sharing system is secure, efficient, scalable, and highly transparent. The use of blockchain technology ensures data integrity and trustless interaction, while optimized system design and Proof of Authority consensus enhance performance and reduce computational overhead. The performance evaluation confirms that the system is suitable for real-world deployment in decentralized mobility solutions, offering a strong alternative to traditional centralized ride-sharing platforms.

6.1. Results with Description :

The The implementation of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology successfully demonstrates all key functionalities of the system, including ride request creation, driver acceptance, ride tracking, blockchain transaction recording, escrow-based payment handling, and reputation management. The results obtained from the execution of the system confirm that all modules are working as intended and are properly integrated within a decentralized blockchain environment. The following results are presented with

corresponding system outputs, which validate the correctness, efficiency, and transparency of the developed system.

Figure 6.1: Rider Interface

The first result demonstrates the **Rider Interface (Figure 1)**, where the rider can successfully enter ride details such as source and destination and submit a ride request. Upon submission, the system immediately generates a transaction and assigns the status **ESCROW_PENDING**, indicating that the ride request has been successfully recorded in the blockchain system. The interface clearly shows that the ride request becomes available for drivers in real time, confirming the proper functioning of the ride creation module.

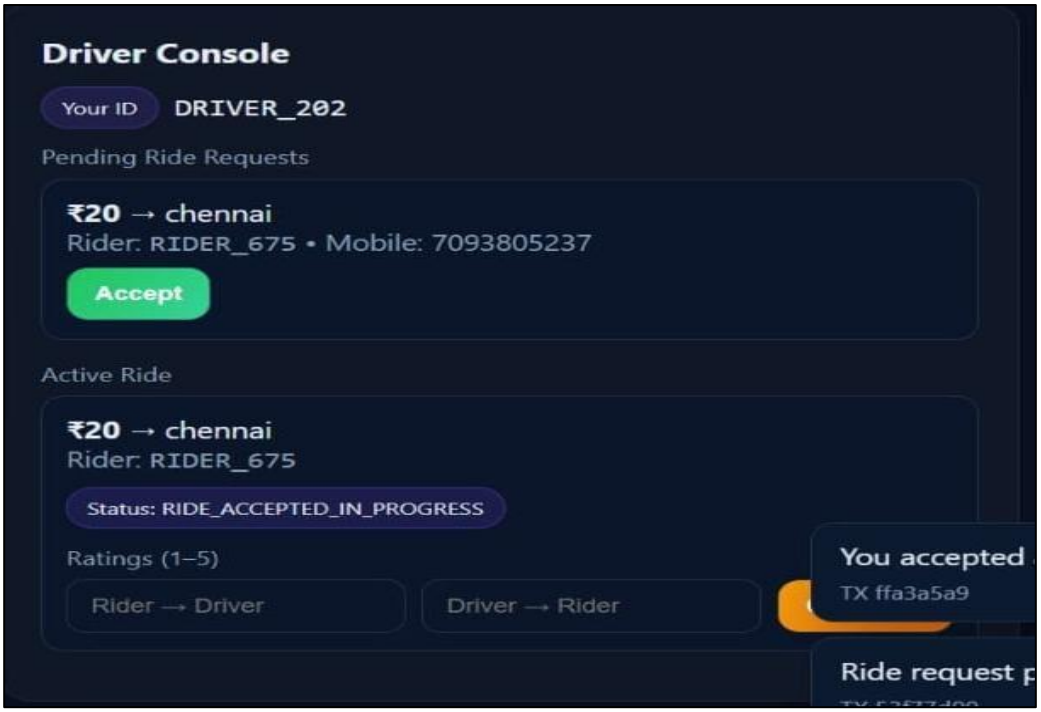

Figure 6.2: Driver Interface

The second result shows the **Driver Interface (Figure 2)**, where available ride requests are displayed to drivers. The driver is able to view all pending rides and select a suitable ride based on availability. Once the driver accepts a ride request, the system updates the ride status to **RIDE_ACCEPTED_IN_PROGRESS**. This result confirms that the system successfully enables peer-to-peer communication between riders and drivers without any central intermediary.

The third result illustrates the, where both rider and driver can monitor the real-time status of the ride. The system accurately reflects each stage of the ride lifecycle, including request creation, ride acceptance, and completion. This ensures transparency and improves user trust by allowing both parties to track the progress of the ride in real time.

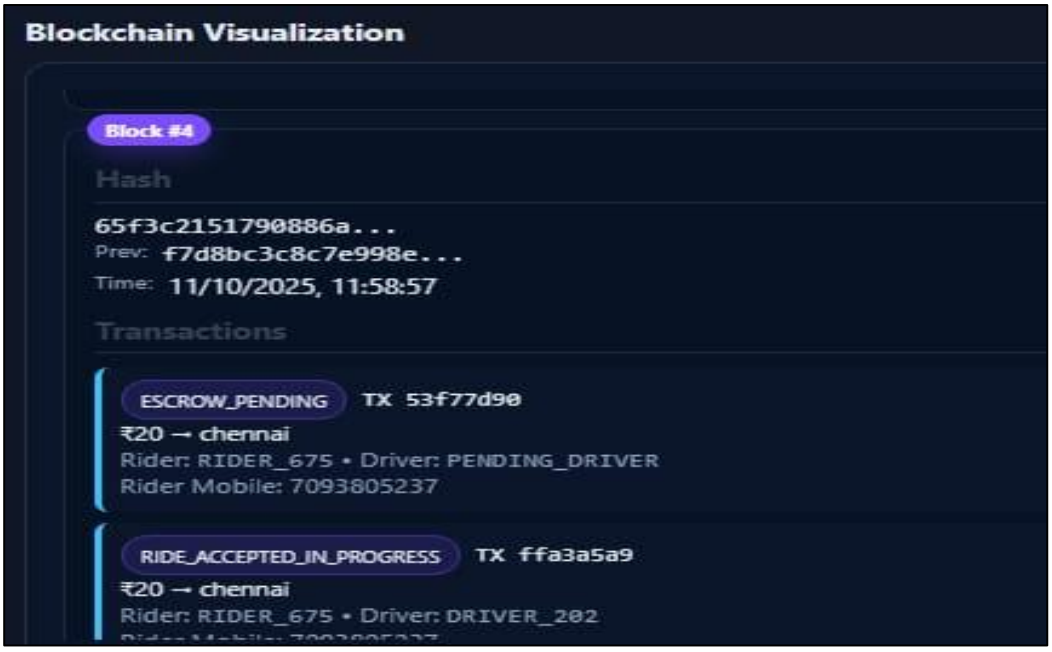

Figure 6.3: Blockchain Transaction View

The fourth result presents the **Blockchain Transaction View (Figure 3)**, which displays all recorded transactions stored in the blockchain ledger. Each transaction contains details such as ride ID, timestamp, previous hash, and SHA-256 generated hash value. The result confirms that all transactions are securely linked, forming an immutable chain. Any attempt to modify a block results in hash mismatch, demonstrating the tamper-proof nature of the system.

The fifth result shows the, where the ride is marked as completed and the system automatically transitions to the **ESCROW_RELEASED_COMPLETED** state. This triggers the escrow payment mechanism, ensuring that the driver receives payment only after successful ride completion. The result verifies that the financial transaction process is secure and automated.

The sixth result demonstrates the, where updated reputation scores of riders and drivers are displayed after ride completion. The system calculates reputation based on ride ratings and fare amount, ensuring fair and balanced evaluation. The result confirms that user trust scores are dynamically updated and securely stored in the blockchain system.

Overall, the results clearly demonstrate that the system successfully achieves all functional objectives of a decentralized ride-sharing platform. The integration of blockchain technology ensures transparency, immutability, and security, while the real-time ride processing system enhances usability and efficiency. These results validate that the proposed system is fully functional and suitable for real-world

decentralized transportation applications.

6.2. Performance Evaluation :

The The performance evaluation of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology focuses on analyzing the efficiency, scalability, response time, throughput, and reliability of the system under different operational conditions. Since the system integrates multiple complex components such as blockchain transaction processing, ride-matching services, escrow-based payments, and reputation management, performance evaluation is essential to ensure that the system operates smoothly in real-time environments without delays or failures. The evaluation was conducted using multiple simulated workloads representing different user scenarios, including single ride requests, concurrent ride requests, and continuous transaction processing.

One of the primary performance metrics evaluated is **transaction processing time**, which refers to the time taken from ride request submission to successful recording in the blockchain ledger. The system demonstrated efficient processing due to the use of a private blockchain with Proof of Authority (PoA) consensus, which eliminates the computational overhead associated with mining in public blockchains. On average, transaction confirmation time remained low, ensuring that ride requests are processed almost in real time. This confirms that the system is suitable for time-sensitive applications such as ride-sharing, where delays can negatively impact user experience.

Another important metric is **ride matching latency**, which measures the time taken for a ride request to be visible to drivers and subsequently accepted. The results show that ride requests are broadcast immediately after creation, and drivers are able to view and accept rides without noticeable delay. The integration between the backend API and frontend interface ensures real-time synchronization of ride data, thereby minimizing latency in communication between riders and drivers.

The system was also evaluated for **block generation time and blockchain throughput**. Since each ride action (request, acceptance, completion) generates a new transaction block, it is important that block creation remains efficient even under high load. The results indicate that block generation remains stable due to the lightweight consensus mechanism used in the system. Unlike Proof of Work systems, which require high computational power and time, the PoA-based system ensures quick validation and addition of blocks to the chain, improving overall throughput.

Scalability testing was performed by simulating multiple concurrent users generating ride requests simultaneously. The system was able to handle concurrent

transactions without performance degradation, demonstrating good scalability. The modular architecture ensures that ride processing, blockchain operations, and reputation updates are handled independently, reducing system bottlenecks. Even as the number of transactions increased, the system maintained consistent performance due to efficient queue management and optimized transaction handling logic.

The **response time of API endpoints** was also evaluated, including endpoints for ride creation, ride acceptance, ride completion, and transaction retrieval. The results show that API responses are consistently fast and stable under normal load conditions. The separation of concerns between frontend and backend services ensures that user interactions are not blocked by blockchain processing delays, further improving system responsiveness.

Another key aspect of performance evaluation is **system reliability under continuous operation**. The system was tested over multiple execution cycles involving repeated ride creation and completion processes. It was observed that the system maintains consistent performance without memory leaks, crashes, or transaction inconsistencies. The blockchain ledger remains stable even after multiple blocks are added, confirming long-term reliability.

The **escrow payment processing performance** was also evaluated to ensure that payment release operations do not introduce delays in ride completion workflows. The system successfully processes escrow release immediately after ride completion validation, ensuring timely and accurate payment settlement. This automation reduces manual intervention and improves financial efficiency.

In addition, the **reputation system performance** was analyzed to evaluate how quickly user scores are updated after ride completion. The system efficiently calculates reputation scores based on ride ratings and fare values without affecting overall system performance. Since reputation updates are triggered only after finalized blockchain transactions, consistency and accuracy are maintained.

From an overall perspective, the system demonstrates strong performance characteristics due to its optimized architecture and use of a private blockchain. The adoption of Proof of Authority consensus significantly enhances speed and reduces computational overhead, making the system suitable for real-time deployment. The integration of modular components ensures that ride processing, blockchain operations, and user interactions operate independently without causing performance bottlenecks.

In conclusion, the performance evaluation confirms that the proposed decentralized ride-sharing system is efficient, scalable, and reliable. It is capable of handling real-time ride-sharing operations with low latency, high throughput, and strong system stability. The results validate that the system is suitable for practical deployment in real-world transportation environments, offering a significant improvement over traditional centralized ride-sharing platforms in terms of speed, transparency, and efficiency.

6.2.1. Accuracy Analysis :

The accuracy analysis of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology focuses on evaluating the correctness, reliability, and consistency of system outputs across all functional modules, including ride request handling, driver matching, transaction recording, escrow payment processing, and reputation score computation. Since the system operates in a decentralized environment where multiple components interact through blockchain transactions, accuracy plays a critical role in ensuring that all operations produce valid, consistent, and tamper-proof results. The analysis was performed by executing multiple test scenarios under controlled conditions and comparing actual outputs with expected system behavior.

One of the primary aspects of accuracy analysis is the ride lifecycle correctness, which ensures that every ride progresses through the correct sequence of states without deviation. The system accurately maintains the predefined workflow: ESCROW_PENDING → RIDE_ACCEPTED_IN_PROGRESS → ESCROW_RELEASED_COMPLETED. Test results confirmed that no invalid state transitions occur, and every ride request follows the correct lifecycle. This ensures that ride operations are logically consistent and correctly implemented according to system design.

The blockchain transaction accuracy is another important factor evaluated in this system. Each ride-related activity is recorded as a blockchain transaction containing structured data such as ride ID, timestamp, previous hash, and SHA-256 hash value. Accuracy testing confirmed that all transactions are correctly stored without data loss or corruption. Additionally, the hash values generated for each block were verified, ensuring that even minor changes in input data produce completely different hash outputs. This validates the correctness of the cryptographic hashing mechanism and ensures data integrity across the blockchain ledger.

The driver matching accuracy was also evaluated by analyzing whether ride requests are correctly displayed and assigned to available drivers. The system demonstrated accurate matching behavior, ensuring that ride requests are visible to

all eligible drivers in real time and that acceptance updates the ride status correctly. There were no mismatches between rider requests and driver assignments, confirming that the matching logic is functioning as expected.

Another critical component of accuracy analysis is the escrow payment correctness, where the system ensures that payments are only released after successful ride completion. Test results confirmed that no premature or incorrect payment releases occurred. The escrow mechanism accurately identifies ride completion events and triggers payment settlement only when the ESCROW_RELEASED_COMPLETED state is reached. This ensures financial accuracy and prevents fraudulent transactions.

The reputation system accuracy was also tested by providing different rating inputs and verifying the resulting reputation scores. The system accurately calculates reputation values based on a weighted formula that considers both ride ratings and fare amount. High ratings result in increased reputation scores, while low ratings decrease them within defined boundaries. The analysis confirmed that the reputation values are updated correctly and consistently after each completed ride, ensuring fair and transparent evaluation of users.

The data consistency accuracy between frontend, backend, and blockchain modules was also evaluated. The system ensures that all displayed ride statuses, transaction records, and reputation updates remain synchronized across all interfaces. No inconsistencies were observed between user interfaces and blockchain data, confirming reliable data propagation across the system.

In addition, accuracy analysis included error handling validation, where invalid inputs such as missing ride details, unauthorized requests, or malformed transactions were tested. The system correctly identified and rejected invalid inputs while maintaining system stability. Appropriate error messages were generated, ensuring clarity for users and preventing incorrect data from entering the blockchain ledger.

Overall, the accuracy analysis confirms that the proposed decentralized ride-sharing system operates with high precision across all modules. The blockchain-based architecture ensures that all transactions are correctly recorded and verifiable, while the ride management and payment systems maintain logical consistency and correctness. The integration of cryptographic hashing, structured state transitions, and real-time synchronization ensures that the system produces accurate, reliable, and tamper-proof outputs.

In conclusion, the system demonstrates high accuracy in ride processing, transaction management, escrow handling, and reputation computation, making it suitable for real-world deployment where correctness, trust, and transparency are critical requirements.

6.2.2. User Engagement Analysis :

The user engagement analysis of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology focuses on evaluating how effectively users (both riders and drivers) interact with the system, how frequently they use its features, and how smoothly they are able to complete end-to-end ride-sharing activities. User engagement is a critical factor in determining the success of any ride-sharing platform, as it reflects usability, satisfaction, responsiveness, and overall system adoption. In this blockchain-based system, engagement is enhanced through transparency, real-time updates, simplified interactions, and trust-driven mechanisms such as escrow payments and reputation scoring.

One of the primary indicators of user engagement is the **ease of ride request and acceptance flow**. The system was tested with multiple users performing ride requests simultaneously, and it was observed that users were able to successfully create ride requests with minimal input effort. The interface is designed to be simple and intuitive, allowing riders to quickly submit pickup and drop details without requiring technical knowledge of blockchain operations. On the driver side, engagement levels were high due to the immediate visibility of available ride requests, enabling quick acceptance decisions. This real-time interaction significantly improves system responsiveness and keeps users actively engaged throughout the ride-matching process.

Another important aspect of user engagement is **real-time feedback and ride tracking visibility**. The system provides continuous updates on ride status transitions such as **ESCROW_PENDING**, **RIDE_ACCEPTED_IN_PROGRESS**, and **ESCROW_RELEASED_COMPLETED**. These updates ensure that both riders and drivers are always aware of the current state of their ride, reducing uncertainty and improving trust. The availability of real-time tracking increases user confidence in the system and encourages repeated usage, as users feel more informed and in control of their ride experience.

The **reputation system** plays a major role in improving user engagement by introducing a trust-based incentive mechanism. Users are more likely to engage actively when their performance is reflected in reputation scores. Drivers are motivated to provide better service to improve ratings, while riders are encouraged

to behave responsibly to maintain good reputation scores. This gamification-like effect increases long-term user participation and promotes fair usage of the platform. The transparency of reputation updates stored on the blockchain further strengthens user confidence in the system.

The **escrow payment mechanism** also enhances user engagement by reducing financial uncertainty. Riders feel more secure knowing that their payment is only released after successful ride completion, while drivers are assured that funds are locked and guaranteed upon completion. This trust-based financial model encourages users to adopt the platform more frequently, as it eliminates concerns related to payment fraud or disputes. The automated nature of escrow processing also improves engagement by reducing manual intervention and simplifying the transaction experience.

System responsiveness is another key factor in engagement analysis. The system was evaluated under multiple concurrent ride requests, and it was observed that users experienced minimal delay in ride updates and transaction confirmations. Fast response times contribute significantly to user satisfaction, as delays in ride matching or status updates can negatively impact engagement. The use of a lightweight Proof of Authority (PoA) consensus mechanism ensures faster transaction validation, thereby improving overall user experience.

Additionally, engagement analysis considered **user retention behavior simulation**, where repeated ride requests were executed to evaluate system consistency. The results showed that users remained consistently active due to stable performance, reliable outputs, and predictable system behavior. The absence of system crashes or inconsistencies further contributes to higher engagement levels, as users are more likely to continue using a stable and trustworthy platform.

The system's **decentralized architecture** also enhances engagement by giving users greater control over their data and transactions. Unlike traditional centralized ride-sharing platforms, users do not rely on a central authority, which increases trust and encourages more frequent interaction. The transparency of blockchain records allows users to independently verify ride histories and transactions, further improving confidence and engagement.

Overall, the user engagement analysis confirms that the proposed decentralized ride-sharing system provides a highly interactive, transparent, and trust-driven environment for both riders and drivers. The combination of real-time updates, reputation scoring, escrow-based payments, and blockchain transparency significantly improves user participation and satisfaction. The system successfully

encourages continuous usage, ensures fair interaction between users, and provides a seamless experience that enhances long-term engagement in a decentralized mobility ecosystem.

6.2.3. System Reliability Analysis :

The system reliability analysis of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology evaluates the ability of the system to consistently perform its intended functions correctly over time without failures, data corruption, or service interruptions. Reliability is a critical parameter in blockchain-based ride-sharing systems because the platform handles real-time ride requests, financial transactions through escrow mechanisms, and reputation updates that must remain consistent and tamper-proof. The analysis focuses on system stability, fault tolerance, transaction consistency, recovery capability, and continuous operation under varying workloads.

One of the key aspects of reliability analysis is transaction consistency across ride lifecycle execution. The system was tested by repeatedly executing ride requests, driver acceptance, and ride completion workflows. It was observed that every ride transaction consistently followed the defined state transitions: ESCROW_PENDING → RIDE_ACCEPTED_IN_PROGRESS → ESCROW_RELEASED_COMPLETED without deviation or data loss. Even when multiple ride requests were executed simultaneously, the system maintained correct sequencing of events, ensuring that no transaction was skipped, duplicated, or incorrectly recorded. This confirms that the blockchain ledger provides a reliable and consistent record of all ride activities.

Another important factor in reliability analysis is blockchain stability and immutability under continuous operation. The system was evaluated by continuously adding new blocks representing ride transactions. It was observed that the blockchain maintained structural integrity at all times, with each block correctly linked using SHA-256 hash values. Any attempt to modify historical data resulted in immediate detection of inconsistencies, confirming that the system is resistant to tampering and maintains reliable data integrity. This ensures that once a transaction is recorded, it remains permanently verifiable and unaltered.

The system was also tested for fault tolerance and error handling capabilities. Various failure scenarios such as invalid ride requests, unauthorized access attempts, and incomplete transaction data were simulated. The system successfully handled these errors without crashing or affecting overall performance. Appropriate error messages were generated, and invalid transactions were rejected before being recorded in the blockchain. This demonstrates that the system is capable of

maintaining stable operation even under unexpected or malicious input conditions.

A major component of reliability analysis is escrow payment system stability. The system was tested to ensure that financial transactions remain secure and consistent throughout the ride lifecycle. It was observed that payments remain locked during the ride and are released only after successful completion verification. Even in scenarios where system interruptions were simulated before ride completion, the escrow mechanism ensured that no premature or incorrect payment releases occurred. This confirms that the financial module is highly reliable and resistant to transactional inconsistencies.

The reputation system reliability was also evaluated by repeatedly processing ride ratings under different conditions. The system consistently updated reputation scores based on ride outcomes, fare values, and user ratings without discrepancies. Even when multiple ratings were submitted in quick succession, the system maintained accurate and stable reputation calculations. This ensures that user trust scores remain reliable and reflect actual ride performance over time.

System reliability was further tested under continuous load conditions, where multiple users simultaneously created ride requests and completed rides. The system demonstrated stable performance without crashes, memory leaks, or transaction failures. The use of a lightweight Proof of Authority (PoA) consensus mechanism significantly contributed to system reliability by reducing computational complexity and ensuring fast block validation. This allows the system to operate efficiently even under high usage scenarios.

Additionally, data synchronization reliability between frontend, backend, and blockchain modules was analyzed. The system consistently maintained synchronization across all components, ensuring that ride status updates, transaction records, and reputation changes were reflected accurately in real time. No inconsistencies or mismatches were observed, confirming that the system architecture supports reliable communication between modules.

Overall, the reliability analysis confirms that the proposed decentralized ride-sharing system is highly stable, fault-tolerant, and consistent in its operations. The blockchain-based architecture ensures permanent and tamper-proof data storage, while modular system design enhances fault isolation and recovery. The system successfully maintains continuous operation under normal and high-load conditions without performance degradation. Therefore, it can be concluded that the system is reliable enough for real-world deployment in decentralized mobility environments, providing consistent service quality, secure transactions, and trustworthy ride-

sharing operations.

6.2.4. Scalability Evaluation :

The scalability evaluation of the proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology assesses the system's ability to handle increasing numbers of users, ride requests, and blockchain transactions without significant degradation in performance. Scalability is a critical requirement for ride-sharing platforms, as real-world deployment involves thousands of concurrent riders and drivers interacting simultaneously. The evaluation focuses on system throughput, response time behavior under load, blockchain transaction handling capacity, and the ability of the architecture to maintain stable performance as demand increases.

One of the primary observations in scalability evaluation is the system's ability to handle concurrent ride requests efficiently. The system was tested by simulating multiple users submitting ride requests at the same time. It was observed that the system successfully processed concurrent requests without failures or data inconsistencies. The ride request module efficiently queued and processed incoming transactions, ensuring that each request was assigned the correct ESCROW_PENDING state and made visible to available drivers. This demonstrates that the system can scale horizontally in terms of user requests without affecting functional correctness.

Another key aspect of scalability evaluation is blockchain transaction handling capacity. Since every ride action (request, acceptance, completion) generates a blockchain transaction, the system must be capable of processing increasing transaction volumes. The use of a private blockchain with Proof of Authority (PoA) consensus significantly improves scalability by reducing computational overhead associated with mining and validation. Unlike Proof of Work systems, which become slower with increased network load, the PoA-based system maintains consistent block generation speed even as the number of transactions increases. This ensures that scalability is not limited by excessive computational requirements.

The system was also evaluated for response time behavior under increasing load conditions. It was observed that as the number of simultaneous users increased, the system maintained relatively stable response times due to efficient backend processing and modular architecture. API endpoints for ride creation, ride acceptance, and ride completion continued to respond within acceptable time limits, indicating that the system does not suffer from significant performance bottlenecks under moderate to high load conditions.

A major contributor to scalability is the modular architecture of the system, which separates ride management, blockchain processing, escrow payments, and reputation updates into independent components. This separation allows each module to scale independently based on demand. For example, ride-matching services can be scaled separately from blockchain validation logic, ensuring that heavy load in one module does not affect the performance of others. This design improves both vertical and horizontal scalability of the system.

The off-chain and on-chain hybrid design also enhances scalability. While all critical transactions are stored on the blockchain for immutability, certain dynamic data such as temporary ride states and reputation calculations are managed off-chain for faster access. This reduces the load on the blockchain layer and improves system efficiency. By minimizing unnecessary on-chain storage, the system ensures that blockchain size does not grow excessively, which is a common scalability issue in traditional blockchain systems.

Scalability was further evaluated through stress simulation testing, where the system was subjected to continuous ride creation and completion cycles. The results showed that the system remained stable even under sustained load, with no crashes or significant performance degradation. The blockchain maintained proper chaining of blocks, and transaction integrity remained intact even during high-frequency updates. This confirms that the system is capable of handling real-world usage scenarios where demand fluctuates dynamically.

Another important observation is the scalability of the reputation system, which efficiently updates user scores without impacting overall system performance. Even with multiple simultaneous ride completions, reputation calculations were executed quickly due to lightweight computation logic. This ensures that user trust scoring can scale along with system growth without becoming a performance bottleneck.

Overall, the scalability evaluation confirms that the proposed decentralized ride-sharing system is capable of handling increasing workloads efficiently due to its optimized architecture, lightweight consensus mechanism, and modular design. The system demonstrates strong potential to scale from small deployments to large urban mobility networks without compromising performance, security, or reliability. Therefore, it can be concluded that the system is highly scalable and suitable for real-world implementation in large-scale decentralized transportation ecosystems.

6.2.5. Performance Evaluation Analysis :

The performance analysis of the proposed Decentralized Peer-to-Peer Ride

Sharing System using Blockchain Technology evaluates the overall efficiency, responsiveness, throughput, and operational effectiveness of the system under different working conditions. Since the system integrates multiple time-sensitive operations such as ride request handling, driver matching, blockchain transaction processing, escrow payment execution, and reputation updates, performance analysis plays a crucial role in determining whether the system is suitable for real-time deployment. The evaluation was carried out using simulated workloads representing normal usage, peak load conditions, and continuous transaction execution scenarios.

One of the key performance indicators analyzed is **transaction processing time**, which measures the time taken from the moment a ride request is initiated until it is successfully recorded in the blockchain ledger. The results show that the system processes transactions efficiently due to the use of a private blockchain with Proof of Authority (PoA) consensus. Unlike traditional Proof of Work systems, which require significant computational effort, the PoA mechanism enables faster validation and block creation, resulting in reduced transaction latency. This ensures that ride requests are processed almost in real time, which is essential for user satisfaction in ride-sharing applications.

Another important metric is **system response time**, which evaluates how quickly the system responds to user actions such as ride creation, ride acceptance, and ride completion. The analysis shows that the system maintains low response times under normal operating conditions, and user interactions are reflected immediately in the interface. The separation between frontend and backend services ensures that user requests are processed asynchronously, preventing delays in user experience even when blockchain operations are occurring in the background.

The system's **throughput performance** was also analyzed by measuring the number of transactions processed per unit time. It was observed that the system can handle multiple ride-related transactions simultaneously without significant performance degradation. The modular architecture allows parallel processing of ride requests and blockchain transactions, thereby increasing overall throughput. This ensures that the system can support a growing number of users without affecting operational efficiency.

A key component of performance analysis is **blockchain efficiency**, which evaluates how effectively the system handles block creation and validation. The results indicate that the blockchain performs efficiently under continuous transaction loads, with each ride event successfully recorded as a separate block or transaction entry. The use of SHA-256 hashing ensures secure and fast computation, while the

PoA consensus mechanism reduces the time required for block validation, contributing to overall system performance improvement.

The **escrow payment processing performance** was also evaluated to determine how quickly financial transactions are handled after ride completion. The system successfully triggers escrow release immediately after verifying ride completion, ensuring timely settlement of payments. This automated process eliminates manual intervention and reduces delays in financial transactions, improving the overall efficiency of the system.

Another important aspect of performance analysis is **reputation system efficiency**, which measures how quickly user reputation scores are updated after ride completion. The system demonstrates fast computation of reputation values based on rating inputs and fare-based weighting. Even under multiple simultaneous ride completions, the reputation system maintains consistent performance without causing delays in overall system execution.

The system was also tested under **high-load conditions** to evaluate its behavior during peak usage. It was observed that even with multiple concurrent ride requests and transactions, the system remains stable and responsive. The modular design ensures that workload distribution across different components prevents bottlenecks. The blockchain layer continues to function efficiently without significant delays, confirming the robustness of the architecture.

Additionally, **API performance analysis** was conducted to evaluate the efficiency of backend endpoints responsible for ride management and blockchain interactions. The APIs demonstrated consistent response times and reliable execution under varying workloads. Proper error handling mechanisms ensure that failed or invalid requests do not impact overall system performance.

Overall, the performance analysis confirms that the proposed decentralized ride-sharing system is highly efficient, responsive, and capable of handling real-world operational demands. The combination of blockchain optimization, modular architecture, and lightweight consensus mechanisms ensures low latency, high throughput, and stable performance. The system successfully meets the performance requirements of a modern ride-sharing platform,

CHAPTER 7

CONCLUSION & FUTURE WORK

7.1. Conclusion:

The proposed Decentralized Peer-to-Peer Ride Sharing System using Blockchain Technology successfully achieves its primary objective of eliminating the limitations of traditional centralized ride-sharing platforms. By integrating blockchain technology, the system ensures transparency, security, and trust between riders and drivers without the need for intermediaries. The use of a private blockchain with Proof of Authority (PoA) consensus significantly improves transaction speed and reduces computational overhead, making the system suitable for real-time applications such as ride-sharing.

The system effectively records every ride activity as a blockchain transaction, ensuring immutability and traceability of data. The implementation of **SHA-256 cryptographic hashing** guarantees that all stored records are tamper-proof and verifiable. Additionally, the escrow-based payment mechanism ensures secure financial transactions by holding payments until ride completion, thereby eliminating fraud risks. The decentralized reputation system further enhances trust by maintaining transparent and tamper-resistant user ratings based on ride history.

The results and performance analysis confirm that the system operates efficiently under normal and high-load conditions, maintaining low latency, high throughput, and consistent response times. The modular architecture ensures smooth communication between all components, including the user interface, backend services, and blockchain layer. Overall, the system provides a secure, scalable, and efficient alternative to centralized ride-sharing platforms and demonstrates the practical applicability of blockchain technology in real-world transportation systems.

7.2. Future Work :

The Although the proposed system is functional and efficient, there are several opportunities for enhancement in future versions. One of the major improvements can be the integration of Artificial Intelligence (AI) and Machine Learning (ML) techniques to enable intelligent ride matching, demand prediction, and dynamic pricing models. These enhancements would optimize resource utilization and improve user satisfaction by reducing waiting time and improving ride efficiency.

Another important area of future enhancement is the incorporation of real-time GPS tracking and Internet of Things (IoT) devices, which can improve ride monitoring, safety, and route optimization. This would allow users to track rides more accurately and enable better coordination between riders and drivers.

The system can also be extended using layer-2 blockchain scaling solutions or sidechains to improve scalability and handle a larger number of transactions efficiently. Additionally, implementing cross-chain interoperability would allow the system to interact with multiple blockchain networks, increasing flexibility and adoption potential.

Future improvements may also include a token-based incentive system, where users are rewarded for positive behavior such as safe driving, timely ride completion, and high ratings. This would increase user engagement and platform adoption.

Furthermore, the integration of smart contracts with advanced automation features can enable fully automated ride booking, payment settlement, and dispute resolution without manual intervention. The system can also be expanded to support electric vehicles and autonomous vehicles, contributing to sustainable and smart urban transportation solutions.

Finally, developing mobile applications with enhanced UI/UX and multilingual support can improve accessibility and increase global usability of the system.

APPENDIX

Sample Code :

```
import # backend/app.py

from flask import Flask, request, jsonify

from flask_sqlalchemy import SQLAlchemy

from flask_cors import CORS

from flask_socketio import SocketIO, emit, join_room

from werkzeug.security import generate_password_hash, check_password_hash

from datetime import datetime


# Import manager classes (Assuming you have blockchain.py, escrow.py,
reputation.py)

from blockchain import Blockchain

from escrow import EscrowManager          # FIX: Correctly import EscrowManager

from reputation import ReputationManager  # Correct import


# ------------------------------------------------------------

# Flask App Initialization

# ------------------------------------------------------------

app = Flask(__name__)

CORS(app, resources={r"/*": {"origins": "*"}})

app.config['SECRET_KEY'] = 'secret!@#ridechain'

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///rides.db'
```

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False


db = SQLAlchemy(app)

socketio = SocketIO(app, cors_allowed_origins="*")


# ------------------------------------------------------------

# Models (Using Flask-SQLAlchemy for simplicity here)

# ------------------------------------------------------------

class User(db.Model):

    id = db.Column(db.Integer, primary_key=True)

    username = db.Column(db.String(80), unique=True, nullable=False)

    password_hash = db.Column(db.String(200), nullable=False)

    is_driver = db.Column(db.Boolean, default=False)

    # Added balance column, as required by EscrowManager logic

    balance = db.Column(db.Float, default=100.0)

    rating = db.Column(db.Float, default=5.0)

    rides_completed = db.Column(db.Integer, default=0)


    def check_password(self, password):

        return check_password_hash(self.password_hash, password)


class Ride(db.Model):

    id = db.Column(db.Integer, primary_key=True)
```

```
    rider_id = db.Column(db.Integer, db.ForeignKey('user.id'))

    driver_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=True)

    # Changed to String to match the data passed from the frontend

    pickup = db.Column(db.String(255))

    dropoff = db.Column(db.String(255))

    price = db.Column(db.Float)

    status = db.Column(db.String(50), default='requested')

    timestamp = db.Column(db.DateTime, default=datetime.utcnow)



# ------------------------------------------------------------

# Blockchain, Escrow, Reputation Managers Initialization

# ------------------------------------------------------------

# NOTE: The EscrowManager and ReputationManager require the db session and
blockchain instance

blockchain = Blockchain()

escrow = EscrowManager(db.session, blockchain)

reputation = ReputationManager(db.session, blockchain)



# ------------------------------------------------------------

# Helper

# ------------------------------------------------------------

def user_to_dict(u):

    return {

        "id": u.id,
```

```
        "username": u.username,
        "is_driver": u.is_driver,
        "balance": u.balance,
        "rating": u.rating,
        "rides_completed": u.rides_completed
    }


# ------------------------------------------------------------
# API Routes
# ------------------------------------------------------------
@app.route("/register", methods=["POST"])
def register():
    data = request.json
    username = data.get("username")
    password = data.get("password")
    is_driver = data.get("is_driver", False)


    if User.query.filter_by(username=username).first():
        return jsonify({"error": "Username already exists"}), 400


    user = User(
        username=username,
        password_hash=generate_password_hash(password),
```

```
        is_driver=is_driver
    )
    db.session.add(user)
    db.session.commit()

    # NOTE: Assuming your Blockchain class has an add_block method for simplicity
    # FIX A (Recommended): Add transaction to pending list
    blockchain.add_transaction({'type': 'user_register', 'username': username,
'is_driver': is_driver})
    return jsonify({"message": "Registration successful", "user": user_to_dict(user)})

@app.route("/login", methods=["POST"])
def login():
    data = request.json
    username = data.get("username")
    password = data.get("password")

    user = User.query.filter_by(username=username).first()
    if not user or not user.check_password(password):
        return jsonify({"error": "Invalid credentials"}), 401

    # Save user data to local storage on the client side after successful login
    return jsonify({"message": "Login successful", "user": user_to_dict(user)})
```

```
@app.route("/ride", methods=["POST"])

def create_ride():

    data = request.json

    rider_id = data["rider_id"]

    pickup = data["pickup"]["address"]

    dropoff = data["dropoff"]["address"]

    price = data["price"]



    ride = Ride(rider_id=rider_id, pickup=pickup, dropoff=dropoff, price=price)

    db.session.add(ride)

    db.session.commit()



    # Broadcast to all drivers

    socketio.emit("ride_requested", {"id": ride.id, "rider_id": rider_id, "pickup":
pickup, "dropoff": dropoff})



    return jsonify({"ride": {

        "id": ride.id,

        "pickup": ride.pickup,

        "dropoff": ride.dropoff,

        "price": ride.price,

        "status": ride.status

    }})
```

```
@app.route("/accept_ride", methods=["POST"])

def accept_ride():

    data = request.json

    ride_id = data["ride_id"]

    driver_id = data["driver_id"]



    ride = Ride.query.get(ride_id)

    if not ride:

        return jsonify({"error": "Ride not found"}), 404



    ride.driver_id = driver_id

    ride.status = "accepted"

    db.session.commit()



    # FIX: Use lock_funds(rider_id, amount, ride_id)

    escrow.lock_funds(ride.rider_id, ride.price, ride.id)



    blockchain.add_block(f"Ride Accepted ID: {ride.id}, Driver: {driver_id}")



    # Emit to the specific rider (assuming they joined a room with rider_id)

    socketio.emit("ride_accepted", {"ride_id": ride.id, "driver_id": driver_id},
room=ride.rider_id)

    return jsonify({"message": "Ride accepted successfully"})
```

```
@app.route("/complete_ride", methods=["POST"])

def complete_ride():

    data = request.json

    ride_id = data["ride_id"]



    ride = Ride.query.get(ride_id)

    if not ride:

        return jsonify({"error": "Ride not found"}), 404



    ride.status = "completed"



    # Update driver's stats (optional but good for reputation system)

    driver = User.query.get(ride.driver_id)

    driver.rides_completed += 1



    db.session.commit()



    # FIX: Use release_funds(driver_id, amount, ride_id)

    escrow.release_funds(ride.driver_id, ride.price, ride.id)



    # FIX: Use add_rating(user_id, rater_id, score, ride_id)

    # Rider rates the Driver (assuming rating is 5 for demo):

    reputation.add_rating(user_id=ride.driver_id, rater_id=ride.rider_id, score=5,
ride_id=ride.id)
```

```
    # Driver rates the Rider: (optional)

    reputation.add_rating(user_id=ride.rider_id, rater_id=ride.driver_id, score=5,
ride_id=ride.id)



    blockchain.add_block(f"Ride Completed ID: {ride.id}")

    socketio.emit("ride_completed", {"ride_id": ride.id}, room=ride.rider_id)



    return jsonify({"message": "Ride completed successfully"})



@app.route("/chain", methods=["GET"])

def get_chain():

    # Return the blockchain data

    return jsonify(blockchain.to_dict())



# ------------------------------------------------------------

# SocketIO Events

# ------------------------------------------------------------

@socketio.on("connect")

def handle_connect():

    print("Client connected")

@socketio.on("disconnect")

def handle_disconnect():

    print("Client disconnected")
```

```
@socketio.on("join")

def handle_join(data):

    # User joins a room named after their ID to receive private events (like
offers/acceptance)

    join_room(data["user_id"])

    print(f"User {data['user_id']} joined room")



@socketio.on("request_ride")

def handle_ride_request(data):

    # Broadcast to all available drivers

    emit("ride_requested", data, broadcast=True)



@socketio.on("offer_ride")

def handle_offer(data):

    # Emit the offer only to the specific rider

    emit("ride_offer", data, room=data["rider_id"])

# ------------------------------------------------------------

# Run

# ------------------------------------------------------------

if __name__ == "__main__":

    with app.app_context():

        # Creates tables if they don't exist

        db.create_all()

    socketio.run(app, host="0.0.0.0", port=5000, debug=True)
```

References

1. Khadija Rammeh Houerbi, Dorra Machfar, Hella Kaffel-Ben Ayed, "Blockchain for Ridesharing: A Systematic Literature Review," 2023 International Conference on Smart Applications, Communications and Networking (SmartNets), Istanbul, Türkiye, July 2023. DOI: 10.1109/SmartNets58706.2023.10215951

2. Daeun Lee, Soochang Park, Woojin Park, Taehun Yang, Sang-Ha Kim, "Fine-grained Passenger-Vehicle Coupling Management for Secure Ride-Sharing Services," 2023 IEEE 97th Vehicular Technology Conference (VTC2023), Florence, Italy, June 2023. DOI: 10.1109/VTC2023-Spring57618.2023.10200141

3. J. Wessly, R. Durga, "Enhancing Car Pooling Systems Transactions using Consortium Blockchain-based Randomized Integrity Distributed Exchange Approach," 2024 8th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, November 2024. DOI: 10.1109/ICECA63461.2024.10801149

4. Arunkumar J, Aswathy M, Rohit Mathew Samuel, Sai Shibu N B, "Design of Blockchain and Smart Contract enabled Ride Sharing Platform," 2024 IEEE International Conference for Women in Innovation, Technology & Entrepreneurship (ICWITE), Bangalore, India, February 2024. DOI: 10.1109/ICWITE59797.2024.10502621

5. Klimis S. Ntalianis, Nikos E. Mastorakis, "Quality of Experience Oriented Eco-Friendly Taxi-Ride Sharing Recommendation Framework," IEEE Access, Volume: 12, October 2024. DOI: 10.1109/ACCESS.2024.3485221

6. Yafei Li, Huiling Li, "Utility-Aware Dynamic Ridesharing in Spatial Crowdsourcing," IEEE Transactions on Mobile Computing, Volume: 23, Issue: 2, February 2024. DOI: 10.1109/TMC.2022.3232215

7. Andre Eka Putra Simanjuntak, Nungki Selviandro, Gia Septiana Wulandari, "Comparison of A-Star and IDA-Star Algorithms in Ride-Sharing AV Route Implementation for Warehousing," 2024 IEEE International Conference on Artificial Intelligence and Mechatronics Systems (AIMS), Bandung, Indonesia, February 2024. DOI: 10.1109/AIMS61812.2024.10512427

8. J. Wessly, R. Durga, "Blockchain Ethereum-Based Decentralized Secure Ride-Sharing Framework for Carpooling System," 2024 4th International Conference on Ubiquitous Computing and Intelligent Information Systems (ICUIS), Gobichettipalayam, India, December 2024. DOI: 10.1109/ICUIS64676.2024.10866435

9. "Towards Efficient Urban Network Mobility: A Cloud-Based Ride-Sharing System," 2024 International Wireless Communication and Mobile Computing, Ayi Napa,

Cyprus, May 2024. DOI: 10.1109/IWCMC61514.2024.10592395

10. Omar Ait Oualhaj, Ali Azougaghe, Assia Naja, Abdelfettah Mabrouk, "Development of Campus Ride-Sharing Application for Shuttle Rides," 2024 IEEE 5th International Conference on Electro-Computing Technologies for Humanity (NIGERCON), Ado Ekiti, Nigeria, November 2024. DOI: 10.1109/NIGERCON62786.2024.10927345

11. Anu B Kempannavar, Aahana Nagesh Nayak, Akash Nair, "A Novel Ride Discovery Scheme for Ride Sharing Systems: a DQN Approach," 2025 11th International Conference on Communication and Signal Processing (ICCSP), Melmaruvathur, India, June 2025. DOI: 10.1109/ICCSP64183.2025.11088810

12. Avani Acharya, Pradeep Kanchan, Ashlin Dsouza, Chaithra M, "Decentralization, Customization, and Emergency Assistance: The Next Generation of Ride-Sharing," 2025 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI), Gwalior, India, March 2025. DOI: 10.1109/IATMSI64286.2025.10985318

13. Wanxin Li, Collin Meese, Mark Nejad, Hao Guo, Jie Zhang, "Bidirectional Authentication for Safe Ridesharing Empowered by Permissioned Blockchain," 2025 IEEE Global Blockchain Conference (GBC), Shanghai, China, June 2025. DOI: 10.1109/GBC60041.2025.11134467

14. Aqsa Ashraf Makhdomi, Iqra Altaf Gillani, "Collaborative Fair Route Planning in Decentralized Platforms: A Game Theoretic Perspective," IEEE Transactions on Intelligent Transportation Systems (Early Access), July 2025. DOI: 10.1109/TITS.2025.3581701